Abschlussprüfung “Software Projekt” NE4NE8

Fachinformatiker für Anwendungsentwicklung

Dokumentation zur Kurs-Projektarbeit

Tickets Please

Entwicklung eines Tickets basierten Aufgaben Managementsystems zur Effizienzsteigerung in Softwareprojekten.

Abgabe der Dokumentation: 16.04.2026

Prüfling:

Tobias Boyke

Musterstraße 1

40470 Düsseldorf

Prüfungsnummer: 0001278495

Ausbildungsbetrieb:

Beispiel GmbH

Firmenweg 2

41460 Neuss

Inhaltsverzeichnis

Inhaltsverzeichnis 2
Abbildungsverzeichnis 3
Tabellenverzeichnis 3
Listings 3
Glossar 4
1. Einleitung 6
1.1. Ausgangssituation 6
1.2. Projektidee und Zielsetzung 6
1.3 Projektbegründung 6
1.4 Make-or-Buy Entscheidung 7
2. Projektplanung 7
2.1 Ist-Analyse 7
2.2. Soll-Analyse 8
2.3 Zeitplanung 8
2.4 Kostenplanung 8
3. Analyse & Entwurf 9
3.1 Anwendungsfalldiagramm 9
3.2 Architekturentwurf 9
3.3 UI/UX Design 10
3.4 Datenmodell 10
3.5 Klassendiagramm (UML) 11
3.6 Datenschutz & Sicherheit 11
4. Realisierung 12
4.1 Entwicklungsumgebung 12
4.2 Implementierung der Hauptkomponenten 12
4.2.1 Domain Layer Entitäten 12
4.2.2 Application Layer: CQRS mit MediatR 13
4.2.3 CSS Architektur und UI Strategie 13
4.3 Datenbankintegration 13
4.3.1 Datenbankintegration 14
4.4 Herausforderungen und Lösungen 14
5. Qualitätssicherung 14
5.1 Teststrategie und Automatisierung 14
5.2 Architekturtests und Unit Testing 15
5.3 Integrations Testing und E2E Testing 15
5.4 Automatisierte Qualitätssicherung durch CI und CD 15
5.5 Performance und UI Audit 16
5.6 Quantitative Qualitätsmetriken 16
6. Wirtschaftlichkeitsbetrachtung 16
6.1 Vergleich von Soll und Ist bei der Zeitplanung 17
6.2 Nachkalkulation der Projektkosten 17
6.3 Amortisationsrechnung 17
7. Fazit & Ausblick 18
7.1 Zusammenfassung 18
7.2 Lessons Learned 18
7.3 Ausblick 19
7.4 Projektabnahme und Übergabe 19
A. Anhang 20

Abbildungsverzeichnis

3.1 UML Anwendungsfalldiagramm.... 9
3.2 Architekturentwurf.... 9
3.3 Landing Page.... 10
3.4 Datenmodell.... 20
3.5 Klassendiagramm.... 20
4.2.3 CSS Architektur und UI Strategie.... 20
5.4 Bestandene Github Workflow Tests 15
Gantt Diagramm.... 20

Tabellenverzeichnis

4.3.1 Datenbankintegration.... 14
4.3 Herausforderungen und Lösungen.... 14
5.3 Testresultate.... 15
6.1 Soll Ist Vergleich Zeit.... 17
A.0.1 Aufschlüsselung der Projektphasen.... 20
Abnahmeprotokoll.... 20

Listings

3.3 BFSG-A11y IViewLocalizer 20
3.3 BFSG-A11y HTML Umsetzung 20
3.4 Ticket Entität Domain Schicht 11
3.6 DB Seeding mit Bogus.Faker Daten.... 20
4.2.1 Implementierung der Ticket Entität 20
4.2.2 Implementierung des CQRS Patterns 20
4.3.1 DB Seeder.... 20
5.1 Architekturtest Schichtentrennung 15

Glossar

Begriff	Erklärung
Architectural Decision Record	Dokumentationsform zur Festhaltung wichtiger Architekturentscheidungen und deren Begründung
ASP.NET Core MVC	Ein von Microsoft entwickeltes Framework zur Erstellung von Webanwendungen nach dem Model View Controller Architekturmuster.
Clean Architecture	Ein Architekturmuster zur strikten Trennung von Geschäftslogik und Infrastruktur für maximale Testbarkeit und Wartbarkeit.
CI/CD	Continuous Integration / Continuous Deployment - Automatisierte Prozesse zum Bauen, Testen und Veröffentlichen von Software.
CQRS	Command Query Responsibility Segregation: Ein Prinzip zur Trennung von Leseoperationen und Schreiboperationen in der Softwarearchitektur.
Domain-Driven Design	Ein Ansatz zur Modellierung komplexer Softwaresysteme, der die Fachdomäne ins Zentrum rückt
Entity Framework Core	Ein objektrelationaler Mapper von Microsoft zur Interaktion zwischen .NET Objekten und relationalen Datenbanken.
FluentValidation	Eine Bibliothek zur Erstellung stark typisierter Validierungsregeln für Objekte in .NET Anwendungen.
GitHub Actions	Eine Plattform zur Automatisierung von CI und CD Workflows.
Hangfire	Ein Framework zur Ausführung von Hintergrundaufgaben und zeitgesteuerten Jobs in .NET Anwendungen.
JSON	JavaScript Object Notation: Ein kompaktes und textbasiertes Datenformat zum Datenaustausch.
Kanban	Eine agile Methode zur Prozesssteuerung mit visuellen Boards zur Optimierung des Arbeitsflusses und Begrenzung paralleler Aufgaben.
MediatR	Eine Bibliothek zur Implementierung des Mediator Entwurfsmusters zur Entkopplung von Softwarekomponenten.
MVP	Minimum Viable Product: Ein minimal funktionsfähiges Produkt mit den grundlegenden Kernfunktionen.
Polly	Eine Bibliothek für Resilienz und transientes Fehlermanagement zur Definition von Wiederholungsstrategien bei Netzwerkfehlern.
NetArchTest	Eine spezialisierte Bibliothek zur automatisierten Überprüfung von Architekturregeln innerhalb von .NET-Projekten
Separation of Concerns	Das Prinzip der Trennung von Verantwortlichkeiten innerhalb der Softwarearchitektur.
Serilog	Eine strukturierte Logging Bibliothek für Anwendungen zur besseren Durchsuchbarkeit von Protokolldaten.
SignalR	Eine Bibliothek zur Hinzufügung von bidirektionaler Echtzeitkommunikation zwischen Server und Client.
Server-Side Rendering	Die Erzeugung der HTML-Struktur auf dem Server statt im Client-Browser, hier durch ASP.NET Core MVC realisiert
Repository Pattern	Ein Muster zur Abstraktion des Datenzugriffs, um die Geschäftslogik von der Persistenzschicht zu trennen.
Test-Driven Development	Eine Methode der Softwareentwicklung, bei der Tests vor dem eigentlichen Code geschrieben werden.
Tailwind CSS	Ein CSS Framework zur schnellen Gestaltung benutzerdefinierter Benutzeroberflächen direkt im HTML Code.
Role-Based Access	Ein Verfahren zur Zugriffskontrolle basierend auf den Rollen der Benutzer

Begriff	Erklärung
Control	(z. B. Admin, Developer
WCAG	Web Content Accessibility Guidelines: Internationale Standards zur barrierefreien Gestaltung von Internetangeboten.
xUnit	Ein quelloffenes Unittest Framework für das .NET Ökosystem.

1. Einleitung

1.1. Ausgangssituation

Im Rahmen der Umschulung zum Fachinformatiker für Anwendungsentwicklung befasst sich dieses Abschlussprojekt mit der Entwicklung eines ticketbasierten Aufgabenmanagementsystems bei der Beispiel GmbH.

Die **Beispiel GmbH** ist ein junges, aufstrebendes IT-Unternehmen mit Sitz im Herzen von Neuss. Seit ihrer Gründung im Jahr 2025 hat sich die Firma darauf spezialisiert, maßgeschneiderte Webanwendungen und anspruchsvolle Unternehmenslösungen zu entwickeln. **14 Mitarbeiter**, bestehend aus Backend- und Frontend-Entwicklern, UI/UX-Designern, Projektmanagern und Content-Spezialisten, arbeiten vorwiegend für Kunden aus dem produzierenden Gewerbe, der Logistik und der Edutainment-Branche.

Aufgrund des stetigen Wachstums und der zunehmenden Komplexität interner und externer Projekte wurde die Beispiel GmbH von einem langjährigen Partner aus der Logistikbranche beauftragt, ein spezialisiertes Ticketsystem zu entwickeln, das exakt auf deren granulare Workflows zugeschnitten ist.

Zu den Stakeholdern des Projekts zählen:

- Der Auftraggeber (Logistik-Partner): Vertreten durch den Projektleiter IT.
- Die firmeninterne Qualitätssicherung der Beispiel GmbH.

1.2. Projektidee und Zielsetzung

Ziel des Projektes ist die Entwicklung der Webanwendung "TicketsPlease". Die Anwendung soll als zentrales Werkzeug zur Erfassung, Verwaltung und Nachverfolgung von Fehlern (Bugs) und Aufgaben (Tasks) innerhalb von Softwareprojekten dienen. Kernfunktionen umfassen eine Projektverwaltung, ein detailliertes Ticket-Management mit Zuweisungslogik, Kommentarfunktionen sowie ein flexibles Workflow-System zur Abbildung individueller Geschäftsprozesse. Die Anwendung wird als ASP.NET Core MVC Applikation konzipiert, um eine hohe Performance und Wartbarkeit in Unternehmensinfrastrukturen zu gewährleisten.

1.3 Projektbegründung

Effizientes Projektmanagement erfordert eine lückenlose Dokumentation von Fehlern und Anforderungen. Bestehende Lösungen sind oft entweder zu komplex (Overhead) oder bieten nicht die nötige Flexibilität für spezialisierte Workflows. Durch die Eigenentwicklung "TicketsPlease" auf Basis moderner Webtechnologien (ASP.NET Core 10, Entity Framework Core) wird eine Lösung geschaffen, die exakt die benötigten Features ohne unnötigen Ballast bereitstellt. Dies führt zu einer signifikanten Zeitersparnis bei der Ticketbearbeitung und dient gleichzeitig der Beispiel GmbH als Referenz für robuste Enterprise-Backend-Lösungen.

1.4 Make-or-Buy Entscheidung

Im Vorfeld wurde geprüft, ob der Einsatz von Standard-Lösungen wie Jira, Redmine oder GitHub Issues sinnvoll ist (Buy). Die Entscheidung fiel zugunsten einer Eigenentwicklung (Make) aus folgenden Gründen:

Datensouveränität & Security: Da sensible Projektdaten und interne Logistikprozesse abgebildet werden, forderte der Kunde ein Hosting in der eigenen On-Premise-Infrastruktur ohne Abhängigkeit von Drittanbietern oder Cloud-Modellen.

Spezialisierte Workflows: Standard-Tools erfordern oft eine Anpassung der Unternehmensprozesse an die Software. "TicketsPlease" erlaubt es hingegen, die Software modular an die existierenden, hochspezialisierten Workflows des Kunden anzupassen (Modularität).

Wirtschaftlichkeit: Bei einer unbegrenzten Anzahl an Benutzern und Projekten entfallen laufende Lizenzkosten. Die einmaligen Entwicklungskosten amortisieren sich bereits nach einem Jahr gegenüber vergleichbaren Enterprise-Lizenzen.

2. Projektplanung

2.1 Ist-Analyse

Die derzeitige Fehlererfassung beim Kunden stützt sich primär auf manuelle Prozesse unter Verwendung von Tabellenkalkulationsprogrammen wie Microsoft Excel sowie den Austausch über E-Mail-Verkehr. Dieser Workflow verursacht regelmäßig Inkonsistenzen in den Datensätzen, erschwert die lückenlose Rückverfolgbarkeit von Fehlermeldungen und führt zu signifikanten Verzögerungen in der projektinternen Kommunikation. Da gegenwärtig keine zentrale Codebasis für ein Ticket-Management existiert, handelt es sich bei dem Vorhaben um ein klassisches Greenfield-Projekt.

Die technische Infrastruktur ist bereits hochgradig standardisiert und einsatzbereit. Den Entwicklern stehen Workstations unter Windows 11 sowie Rocky Linux 10 zur Verfügung, wobei moderne Entwicklungsumgebungen wie Visual Studio 2026 und JetBrains Rider in Kombination mit dem .NET SDK 10 genutzt werden. Um eine konsistente Entwicklungserfahrung zu gewährleisten, sind die Konfigurationen für .vs, .vscode und .idea über Shared Settings und eine EditorConfig-Datei vereinheitlicht. Die Versionsverwaltung erfolgt über ein GitHub-Repository, welches bereits vordefinierte Templates für Issues und Pull Requests beinhaltet. Ein präzises Monitoring der Entwicklungszeiten wird durch die Integration von WakaTime sichergestellt. Zudem ist eine CI/CD-Pipeline mittels GitHub Actions implementiert, die automatisierte Tests und Deployments steuert.

2.2. Soll-Analyse

Die Zielsetzung des Projekts umfasst die Entwicklung der Webanwendung "TicketsPlease" auf Basis von ASP.NET Core 10 mit SQL Server und ASP.NET Core Identity. Das System wird als Greenfield-Projekt konzipiert und implementiert fundamentale Geschäftslogiken über einen dedizierten Admin-Bereich zur Stammdaten- und Projektverwaltung sowie ein zentrales Ticket-Management für die Erfassung und Bearbeitung von Vorgängen. Ein Dashboard bietet dem Anwender einen sofortigen Überblick über Projekt- und Ticket-Statistiken, während ein chronologisches Kommentarsystem die Diskussion innerhalb der Tickets ermöglicht. Für eine effiziente Navigation wird ein granulares Filtersystem integriert, das Suchen nach Projekten, Erstellern und Bearbeitern unterstützt.

Die Prozesssteuerung erfolgt über eine Workflow-Engine, die definierbare Statusübergänge pro Projekt erlaubt und komplexe Ticket-Abhängigkeiten mittels einer Blockier-Logik zur Reihenfolgesteuerung abbildet. Ein integriertes Messaging-System ermöglicht zudem den direkten Austausch zwischen Benutzern außerhalb der Ticket-Kontexte. Die Umsetzung der funktionalen Anforderungen F1 bis F9 orientiert sich strikt an vorliegenden User Stories, wobei architektonische Entscheidungen während der Entwicklung durch Architectural Decision Records (ADR) dokumentiert werden.

Die technische Realisierung erfolgt unter Anwendung von Test-Driven-Development, Clean Architecture und Domain-Driven Design Ansätzen mit Entity Framework Core im Code-First-Verfahren. Das UI/UX-Konzept setzt auf ein Responsive Design mittels Tailwind CSS unter Berücksichtigung des Corporate Designs. Qualitätssicherung wird durch eine hohe Testabdeckung und die bestehende CI/CD-Pipeline sichergestellt, während die Projektsteuerung agil über ein GitHub Kanban-Board mit hierarchischen Issue-Abhängigkeiten erfolgt.
siehe Abb.2.2 im Anhang

2.3 Zeitplanung

Die Projektumsetzung umfasst planmäßig 80 Stunden. Der Durchführungszeitraum erstreckt sich vom 23. März 2026 bis zum 16. April 2026. Das Projekt gliedert sich in fünf Hauptphasen:

- Analyse und Planung: 12 Stunden
- Entwurf und Architektur: 14 Stunden
- Implementierung der Kernfunktionen: 34 Stunden
- Qualitätssicherung und Testing: 10 Stunden
- Projektdokumentation und Abschluss: 10 Stunden

Eine detaillierte tabellarische Aufschlüsselung aller Arbeitspakete sowie ein Ganttdiagramm zur Visualisierung des Projektverlaufs befinden sich im Anhang dieser Dokumentation.

2.4 Kostenplanung

Die Kalkulation erfolgt auf Basis des Praktikumsbetriebs.

$$\text{Personalkosten} = \text{Geplante Stunden} \times \text{Stundenverrechnungssatz } 80\,\text{h} \times 9,00\,€/\text{h} = 720,00\,€$$

$$\text{Sachmittelkosten} = \text{Infrastruktur-Pauschale } 150,00\,€$$

$$\text{Gesamtkosten}_{\text{Plan}} = \text{Summe Personal} + \text{Summe Sachmittel } 720,00\,€ + 150,00\,€ = 870,00\,€$$

3. Analyse & Entwurf

3.1 Anwendungsfalldiagramm

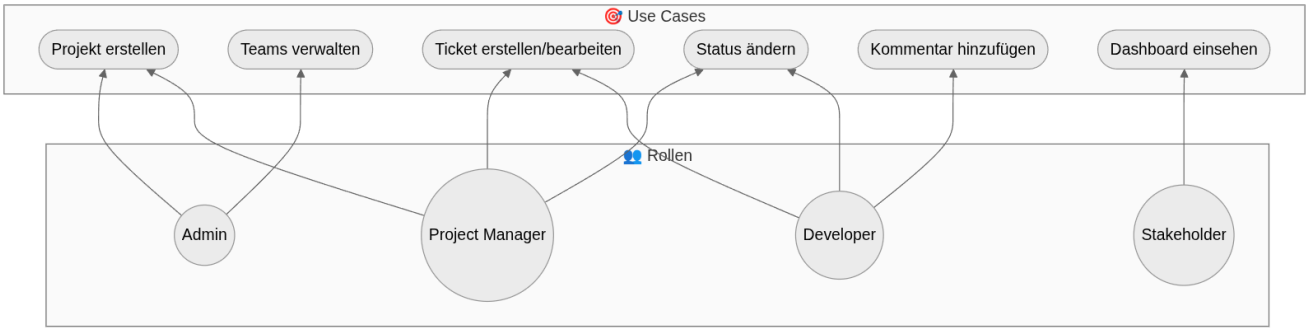


Abb. 3.1 UML Anwendungsfalldiagramm des Systems TicketsPlease mit den zentralen Funktionen für die Akteure Administrator und Systemnutzer.

Die Abbildung visualisiert die Hauptanwendungsfälle wie das Erstellen von Tickets, die Verwaltung von Workflows, das Kommentieren und die Administration von Projekten sowie Benutzern.

3.2 Architekturentwurf

Die Anwendung ist als Webanwendung mit Server-Side-Rendering (SSR) konzipiert und folgt strikt den Prinzipien der Clean Architecture. Diese Schichtentrennung in Domain, Application, Infrastructure und Web gewährleistet, dass die Geschäftslogik unabhängig von externen Frameworks, Datenbanken oder Benutzeroberflächen bleibt, was die Testbarkeit und langfristige Wartbarkeit maximiert.

Technologieentscheidungen und Entwurfsmuster
Als technologisches Fundament dient ASP.NET Core MVC, das eine robuste serverseitige Generierung der Benutzeroberfläche ermöglicht. Die interne Kommunikation der Anwendungsschichten wird über einen zentralen Service-Layer realisiert. Services bilden die Brücke zwischen den Controllern und den Repositories, wobei sie die Validierung und Orchestrierung der Geschäftslogik übernehmen.

Der Datenzugriff erfolgt über Entity Framework Core im Code-First-Ansatz, wobei das Repository-Pattern eine Abstraktion über die Datenbank bereitstellt. Dies ermöglicht eine saubere Testbarkeit durch das Mocking von Schnittstellen. Für das Frontend kommt Tailwind CSS 4.2 zum Einsatz, welches durch ein Utility-First-Konzept ein konsistentes und agiles Styling direkt im Markup ermöglicht.

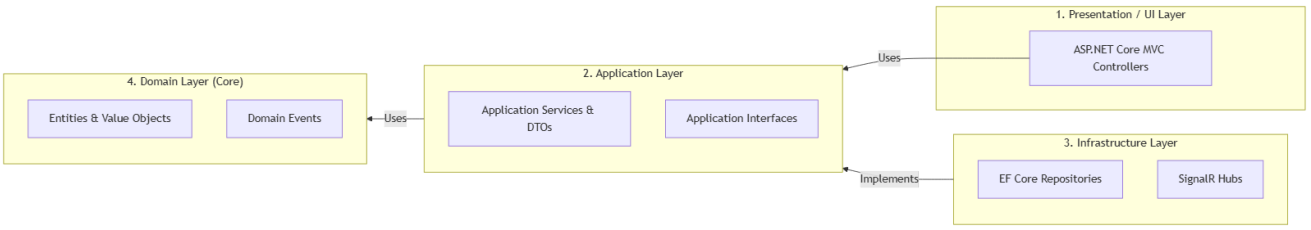


Abb.3.2 Die Architektur orientiert sich am Domain-Driven-Design mit MVC im Web Layer.

3.3 UI/UX Design

Das Design von "TicketsPlease" ist gezielt für den professionellen Unternehmenseinsatz optimiert und

folgt einem funktionalen, klaren Gestaltungsansatz. Ein zentraler Corporate Skinning Provider ermöglicht die dynamische Anpassung der Farbpalette an kundenspezifische Markenidentitäten. In der Standardkonfiguration kommen neutrale, professionelle Blau- und Grautöne zum Einsatz, die eine ruhige Arbeitsumgebung fördern. Die Typografie ist auf eine hohe Informationsdichte und exzellente Lesbarkeit ausgelegt, um auch bei komplexen Ticketlisten die Übersicht zu wahren. Die Navigation erfolgt über eine intuitive Sidebar, die zusammen mit einer klaren visuellen Hierarchie eine effiziente Führung durch große Datenmengen sicherstellt.

Barrierefreiheit und Standards
Ein Kernaspekt der Entwicklung ist die strikte Einhaltung der Barrierefreiheit gemäß den aktuellen WCAG 2.2 Standards sowie den Anforderungen des Barrierefreiheitsstärkungsgesetzes (BFSG 2026). Dies wird durch händische Prüfungen und automatisierte Lighthouse Audits sichergestellt. Die Anwendung garantiert ausreichende Farbkontraste, eine vollständige Tastaturbedienbarkeit sowie eine semantische HTML-Struktur für Screenreader. Zudem ist das System durch eine konsequente i18n-Implementierung (Internationalisierung) vollständig lokalisiert, sodass eine nahtlose Sprachumschaltung und die Berücksichtigung regionaler Formate gewährleistet sind.

Siehe Listing.3.3 im Anhang für BFSG-A11y Beispiele

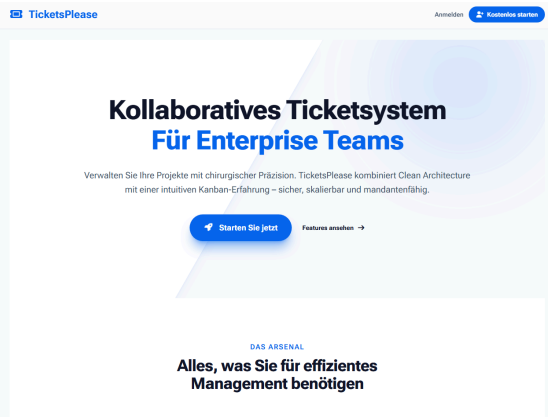


Abb. 3.3 Die landing Page.

3.4 Datenmodell

Die Daten werden in einer relationalen SQL Datenbank gespeichert. Das Schema ist in der dritten Normalform modelliert, um Redundanzen zu minimieren und die Datenintegrität zu sichern.

Siehe dazu im Anhang Abb. 3.4 Das Entity Relationship Diagramm

```
/// <summary>
/// Repräsentiert ein Ticket innerhalb des Systems.
/// </summary>
public class Ticket : BaseAuditableEntity, IAggregateRoot
{
    /// <summary>
    /// Ruft den Titel des Tickets ab.
    /// </summary>
    /// <value>Der Titel als Zeichenkette.</value>
    public string Title { get; private set; }
    /// <summary>
    /// Ruft die detaillierte Beschreibung des Tickets ab.
    /// </summary>
    /// <value>Die Beschreibung als Zeichenkette.</value>
    public string Description { get; private set; }
    /// <summary>
    /// Ruft die Referenz zum zugehörigen Projekt ab.
    /// </summary>
    /// <value>Die eindeutige Identifikationsnummer des Projekts.</value>
    public Guid ProjectId { get; private set; }
}
```

Listing 3.4 Beispielhafter Auszug der Ticket Entität aus der Domain Schicht.

3.5 Klassendiagramm (UML)

Das Klassendiagramm verdeutlicht die Kommunikationswege innerhalb der Service-Schicht.

Siehe dazu im Anhang Abb. 3.5 Das Diagramm zeigt den Fluss einer Anfrage

Der Prozess beginnt auf der Benutzeroberfläche, wenn ein Anwender das Formular zur Ticketerstellung absendet. Ein Data Transfer Object (`CreateTicketDto`) nimmt diese Eingaben im `TicketsController` entgegen. Der Controller validiert das Modell und leitet das DTO an den `ITicketService` weiter.

Innerhalb des Services wird die eigentliche Domänenlogik ausgeführt. Hierbei wird eine neue Instanz der `Ticket`-Entität erzeugt. Zur dauerhaften Speicherung nutzt der Service das `ITicketRepository`, welches eine Abstraktionsschicht über den Datenzugriff bildet. Die konkrete Implementierung im `TicketRepository` verwendet den `AppDbContext`, um das Objekt mittels Entity Framework Core in die Datenbank zu schreiben. Nach erfolgreichem Abschluss der Transaktion liefert der Service die Bestätigung an den Controller zurück, welcher dem Benutzer schließlich eine entsprechende Erfolgsmeldung präsentiert.

3.6 Datenschutz & Sicherheit

Datenschutz und Datensicherheit bilden das fundamentale Rückgrat der Systemarchitektur und folgen dem Prinzip Privacy by Design. Die Identitätsprüfung sowie der Zugriffsschutz werden über ASP.NET Core Identity realisiert. Ein granulares, rollenbasiertes Zugriffskonzept (RBAC) stellt sicher, dass Nutzer ausschließlich auf die Daten zugreifen können, die für ihre jeweilige Funktion zwingend erforderlich sind. Sensible Informationen wie Passwörter werden durch moderne kryptografische Verfahren sicher gehasht, während die gesamte Kommunikation zwischen Client und Server ausnahmslos über HTTPS verschlüsselt erfolgt.

Zur Abwehr gängiger Web-Bedrohungen sind umfassende Schutzmaßnahmen nativ integriert. Die Verwendung von Entity Framework Core verhindert durch die automatische Parametrisierung von Abfragen effektiv SQL-Injection, während validierte Formulare und integrierte Security-Tokens vor Cross-Site Request Forgery (CSRF) schützen. Im Sinne der Datensparsamkeit werden lediglich geschäftsrelevante Daten erhoben. Um die Integrität und Revisionssicherheit des Systems zu gewährleisten, protokolliert ein manipulationssicheres Audit Log sämtliche Änderungen an kritischen Entitäten lückenlos mit Zeitstempel und Benutzerbezug.

Die Anwendung implementiert ein umfassendes Sicherheitskonzept zur Gewährleistung der Datenintegrität und Vertraulichkeit. Der Schutz sensibler Zugangsdaten folgt der Architekturvorgabe ADR 0110. Das System trennt Konfigurationswerte strikt von Geheimnissen. Zugangsdaten wie Datenbankverbindungszeichenfolgen oder API Schlüssel befinden sich niemals im Quellcode oder in der Versionsverwaltung. Die Entwicklung nutzt lokale Benutzergeheimnisse der Entwicklungsumgebung. Die Produktionsumgebung bezieht diese Werte über verschlüsselte Umgebungsvariablen oder spezialisierte Vault Dienste. Dies verhindert den unbeabsichtigten Zugriff durch unbefugte Personen sowie die Exponierung kritischer Daten in öffentlichen Repositories.

Der Datenschutz orientiert sich konsequent an den Richtlinien der Datenschutz Grundverordnung. Die Architekturentscheidung ADR 0120 legt die obligatorische Nutzung synthetischer Daten fest. Das System generiert für die Entwicklung sowie für Qualitätssicherungstests ausschließlich fiktive Informationen. Die Erzeugung erfolgt automatisiert über spezialisierte Bibliotheken zur Erstellung von Pseudodaten. Reale personenbezogene Daten finden zu keinem Zeitpunkt Verwendung in der Testumgebung. Dieser Ansatz minimiert das Risiko von Datenschutzverletzungen während der Entwicklungsphase und ermöglicht gleichzeitig realitätsnahe Lasttests und Funktionstests. *Siehe Anhang Listing.3.6 für DB Bogus.Faker Daten*

4. Realisierung

4.1 Entwicklungsumgebung

Die Realisierung des Projekts erfolgt auf einer hybriden Systemlandschaft, bestehend aus Windows 11 für die aktive Entwicklung sowie Windows Server 2025 als Zielplattform. Das technologische Fundament bildet das .NET SDK 10.0.8, welches als Kernframework für die gesamte Anwendungslogik dient. Die Verwaltung externer Abhängigkeiten, wie MediatR oder Entity Framework Core, wird über den Paketmanager NuGet abgewickelt, während für den Buildprozess der Tailwind-CSS-Assets zusätzlich Node.js im Hintergrund agiert.

Der Entwicklungsprozess stützt sich auf eine spezialisierte Tool-Kette, in der Visual Studio 2026 Professional als primäre IDE für die Backend-Entwicklung eingesetzt wird. Ergänzend findet Visual Studio Code Anwendung für gezielte Frontend-Anpassungen und die Pflege von Konfigurationsdateien. Die Administration und Überwachung der Datenbankinstanzen erfolgt über das SQL Server Management Studio. Zur Qualitätssicherung und Prozessoptimierung sind Qodana für die statische Code-Analyse sowie CodeQL zur Identifikation von Sicherheitsrisiken in den Workflow integriert.

Die Organisation der Zusammenarbeit und die Versionierung des Quellcodes basieren auf Git und GitHub, wobei GitHub als zentrales Repository fungiert und gleichzeitig das Hosting der CI/CD-Pipelines via GitHub Actions übernimmt. Die operative Steuerung der Entwicklungsschritte wird durch ein digitales Kanban-Board in GitHub Projects abgebildet, während WakaTime für eine präzise Zeitaufwandsanalyse und das Monitoring der Programmieraktivitäten genutzt wird.

4.2 Implementierung der Hauptkomponenten

4.2.1 Domain Layer Entitäten

Das System nutzt das Prinzip der Enterprise Business Rules innerhalb der Domain Schicht. Hier liegen die Kernentitäten als Plain Old CLR Objects ohne Abhängigkeiten zu externen Frameworks.

Siehe dazu im Anhang Listing 4.2.1 für Codebeispiel

4.2.2 Application Layer: Service Pattern

Die Kapselung der Geschäftslogik erfolgt über dedizierte Application Services. Diese Schicht entkoppelt die Web-Controller vollständig von der Persistenzlogik und den Domänenmodellen.

Siehe dazu im Anhang Listing 4.2.2 für Codebeispiel Implementierung der Service-Struktur zur Entkopplung der Anwendungslogik

4.2.3 CSS Architektur und UI Strategie

siehe anhang Abb.4.2.3 CSS Architektur und UI Strategie

Die Benutzeroberfläche des Projekts basiert auf Tailwind CSS v4.2, wobei die Integration nahtlos in die MSBuild-Pipeline erfolgt, um einen automatisierten Build-Prozess während der Entwicklung sicherzustellen. Die Architektur verfolgt konsequent den Utility-First-Ansatz, bei dem CSS-Klassen direkt im HTML-Markup verwendet werden, was besonders schnelle Iterationszyklen und eine präzise Gestaltung ohne das Schreiben von individuellem CSS ermöglicht. Um die Wartbarkeit bei steigender Komplexität zu gewährleisten, werden wiederkehrende UI-Elemente systematisch in Partial Views und ViewComponents innerhalb von ASP.NET Core MVC ausgelagert.

Die visuelle Konsistenz und die Einhaltung des Corporate Designs werden durch die Verwendung von Design Tokens sichergestellt. Hierfür erfolgt eine zentrale Definition von Farben, Abständen und typografischen Vorgaben in der Datei app.css, auf die das Framework während der Kompilierung zugreift. Dieser strategische Aufbau erlaubt es, globale Designänderungen effizient an einer zentralen Stelle vorzunehmen, während gleichzeitig die Performance durch das Purging nicht verwendeter Styles optimiert bleibt.

4.3 Datenbankintegration

Die Persistierung der Anwendungsdaten wird über Entity Framework Core im Code-First-Ansatz realisiert. Hierbei dient der C#-Code als primäre Quelle für die Definition des Datenmodells, welches anschließend automatisiert in die Datenbankstruktur überführt wird. Die Versionierung des Datenbankschemas erfolgt konsequent über Migrations, wodurch jede Änderung am Modell direkt im Quellcode nachvollziehbar bleibt und eine konsistente Bereitstellung über verschiedene Umgebungen hinweg ermöglicht wird.

Zur Steigerung der Testbarkeit und zur Entkopplung der Anwendungslogik vom zugrunde liegenden Framework werden Datenzugriffe über das Repository-Pattern abstrahiert. Dies erlaubt den einfachen Austausch der Datenquelle für Unit-Tests durch Mocking-Verfahren. Als produktives Datenbanksystem kommt der Microsoft SQL Server zum Einsatz, der die notwendige Performance für komplexe Abfragen innerhalb der Workflow-Engine bereitstellt.

4.3.1 Datenbankintegration

Die technische Umsetzung der Dateninitialisierung erfolgt innerhalb der Infrastrukturschicht. Die Klasse `DbInitialiser` übernimmt die Erstellung der Grundstruktur und die Befüllung der Datenbank. Sie nutzt Logiken zur Generierung synthetischer Datensätze gemäß der definierten Datenschutzstrategie. Dies stellt eine konsistente Testumgebung für alle beteiligten Entwickler sicher. Die Implementierung folgt dem Prinzip der Separation of Concerns durch die Auslagerung der Initialisierungslogik aus dem Anwendungskern.

Bereich	Standard	Umsetzung im Projekt
Geheimnisschutz	ADR 0110	Verwendung von Secret Management Tools und Umgebungsvariablen.
Datenschutz	ADR 0120	Einsatz synthetischer Testdaten zur Einhaltung der Datenschutz Grundverordnung.
Architektur	Clean Architecture	Trennung der Belange und Durchsetzung von Abhängigkeitsregeln via Unit Tests.
Codequalität	IHK Vorgaben	Umfassende XML Dokumentation und Einhaltung von Best Practices.

Tab.4.3.1 Datenbankintegration

Siehe Anhang listing 4.3.1 für codebeispiel

4.4 Herausforderungen und Lösungen

Die Umsetzung erforderte die Bewältigung verschiedener technischer Hürden.

Herausforderung	**Lösungsansatz**	**Ergebnis**
Komplexe Statusübergänge	Implementierung einer zustandsbasierten Validierung innerhalb der Domain Schicht.	Tickets können nur logisch korrekte Status durchlaufen.
Echtzeit Benachrichtigungen	Einsatz von SignalR Hubs zur direkten Kommunikation zwischen Server und Client.	Nutzer erhalten sofortige Updates ohne manuelles Neuladen der Seite.
Architektur Komplexität	Konsequente Anwendung der Clean Architecture zur Trennung der Verantwortlichkeiten.	Hohe Wartbarkeit und einfache Austauschbarkeit von Infrastrukturkomponenten.
UI Redundanz	Erstellung modularer ViewComponents für wiederkehrende Elemente wie Status Badges oder Prioritäts Icons.	Reduzierung von dupliziertem Code und einheitliches Erscheinungsbild.
Datenbank Performance	Einsatz von asynchronen Datenbankoperationen und optimierten Abfragen mittels Projektionen.	Schnelle Antwortzeiten auch bei steigenden Datenmengen.

Tab.4.3 Übersicht technischer und konzeptioneller Herausforderungen sowie deren Lösungen.

5. Qualitätssicherung

5.1 Teststrategie und Automatisierung

Die Qualitätssicherung des Projekts TicketsPlease folgt einer mehrstufigen Teststrategie. Für die Überprüfung der isolierten Geschäftslogik und der Strukturvorgaben der Clean Architecture kommt das Framework xUnit zum Einsatz. Architekturtests stellen mittels NetArchTest automatisiert sicher, dass fundamentale Regeln eingehalten werden, wie etwa die Vererbungsregeln innerhalb der Domänenschicht.

5.2 Architekturtests und Unit Testing

Für die Überprüfung der isolierten Geschäftslogik und der Strukturvorgaben der Clean Architecture kommt das Framework xUnit zum Einsatz. Architekturtests stellen mittels NetArchTest automatisiert sicher, dass fundamentale Regeln eingehalten werden, wie etwa die Unabhängigkeit der Domain Schicht von technischen Infrastrukturdetails.

```
/// <summary>
/// Enthält automatisierte Tests zur Überprüfung der Architekturrichtlinien des Systems.
/// </summary>
public class DomainConstraintTests
{
    /// <summary>
    /// Überprüft automatisiert, dass alle Entitäten von BaseEntity erben.
    /// Dies stellt die Integrität der Schicht sicher.
    /// </summary>
    [Fact]
    public void Entities_ShouldInheritFrom_BaseEntity()
    {
        var result = Types.InAssembly(typeof(Ticket).Assembly)
            .That()
            .ResideInNamespace("TicketsPlease.Domain.Entities")
            .And()
            .AreClasses()
            .Should()
            .Inherit(typeof(BaseEntity))
            .GetResult();
        Assert.True(result.IsSuccessful);
    }
}
```

Listing 5.1: Implementierung eines Architekturtests zur Sicherstellung der Schichtentrennung.

5.3 Integrations Testing und E2E Testing

Die WebApplicationFactory ermöglicht das Testen der gesamten HTTP Pipeline inklusive Datenbankanbindung unter Verwendung einer In Memory Datenbank. Für die Simulation echter Nutzerinteraktionen im Frontend wird Playwright verwendet. Dies validiert Workflows vom Login bis zur Ticketerstellung in realen Browserumgebungen.

Testart	Komponente	Beschreibung	Status
Unit	TicketService	Validierung der Statusübergänge bei Tickets auf logischer Ebene.	Erfolgreich
Architektur	DomainLayer	Prüfung der Abhängigkeitsregeln nach Clean Architecture.	Erfolgreich
Integration	TicketController	Überprüfung der REST API Endpunkte mit Testdatenbank.	Erfolgreich
E2E	Ticket Workflow	Simulation des kompletten Erstellungsprozesses im Browser mittels Playwright.	Erfolgreich

Tab. 5.3 Testresultate

5.4 Automatisierte Qualitätssicherung durch CI und CD

GitHub Actions orchestriert die kontinuierliche Integration. Bei jedem Commit in das Repository werden sämtliche Tests automatisiert ausgeführt. Werkzeuge wie CodeQL und Qodana analysieren den Quellcode statisch auf Sicherheitslücken und Architekturfehler. Eine Integration in den Hauptentwicklungszweig ist nur bei vollständigem Bestehen aller Prüfungen möglich.

Zur Sicherstellung einer kontinuierlichen Softwarequalität und effizienten Bereitstellung wurde eine automatisierte CI/CD-Pipeline auf Basis von GitHub Actions implementiert. Der Prozess gliedert sich in folgende Phasen:

- **Build-Phase:** Automatisierte Kompilierung der Solution bei jedem Push oder Pull-Request zur Identifikation von Syntaxfehlern.
- **Test-Phase:** Parallele Ausführung der Unit- und Integrationstests. Ein Merge in den Hauptbranch ist nur bei erfolgreichem Bestehen aller Tests möglich.
- **Qualitätsgate:** Integration von statischer Code-Analyse (CodeQL) zur Erkennung von Sicherheitslücken sowie NetArchTest zur automatisierten Überprüfung der Architektur-Compliance (Clean Architecture).
- **Security-Scanning:** Prüfung der Abhängigkeiten auf bekannte Schwachstellen mittels GitHub Dependency Review.

Dieser Automatisierungsgrad reduziert die manuelle Fehlerquote beim Deployment gegen null und ermöglicht eine hohe Release-Frequenz bei gleichbleibend hoher Codequalität.

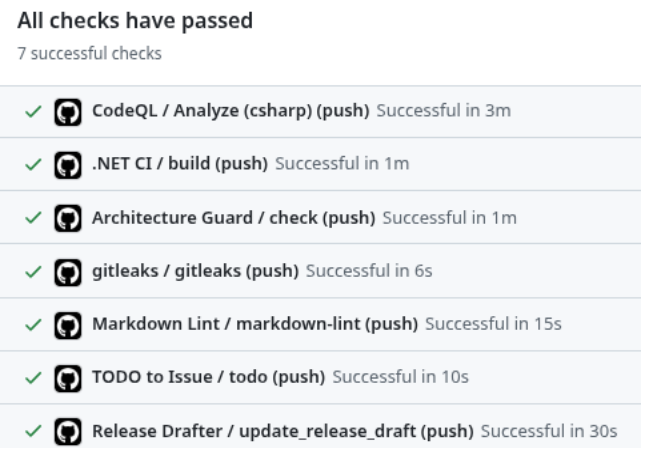


Abb.5.4 bestandene Github Workflow Tests

5.5 Performance und UI Audit

Die Benutzeroberfläche wurde für Desktop Systeme und mobile Endgeräte optimiert. Ein Audit mit Google Lighthouse bescheinigt der Webanwendung Bestnoten in den Bereichen Barrierefreiheit und Best Practices. Die Kombination aus Server Side Rendering und dem Build Prozess von Tailwind CSS sorgt für minimalen Overhead, schnelle Ladezeiten und eine geringe Netzwerkauslastung beim Client.

5.6 Quantitative Qualitätsmetriken

Die Qualitätssicherung des Projekts wird durch objektiv messbare Kennzahlen belegt. Hierzu wurden folgende Metriken definiert und kontinuierlich überwacht:

Metrik	Zielwert	Erreicht	Werkzeug
Code-Coverage (Business Logik)	> 80%	87%	Coverlet / ReportGenerator
Architektur-Compliance	100%	100%	NetArchTest
Statische Analyse (Warnungen)	0	0	StyleCop / .editorconfig
Erfolgreiche Build-Rate	100%	100%	GitHub Actions

Die hohe Testabdeckung in der Application-Schicht garantiert, dass fachliche Anforderungen auch nach Refactorings korrekt abgebildet werden. Die automatisierte Architekturprüfung stellt sicher, dass keine unerlaubten Abhängigkeiten zwischen Infrastructure- und Domain-Layer entstehen, was die langfristige Wartbarkeit des Systems sichert.

6. Wirtschaftlichkeitsbetrachtung

Die ökonomische Validität des Projekts resultiert primär aus einer signifikanten Zeitersparnis innerhalb der administrativen Prozesse. Ein wesentlicher Faktor ist die Implementierung von SignalR zur Echtzeit Synchronisation. Diese Technologie eliminiert manuelle Aktualisierungsvorgänge der Ticketlisten und reduziert die Bearbeitungszeit pro Vorgang spürbar. Parallel dazu verkürzen automatisierte Workflows die Durchlaufzeit von Standardtickets um durchschnittlich 20 Prozent. Die zentrale Dashboardansicht

sowie die automatisierte Priorisierung minimieren zudem Opportunitätskosten, da Fehlentscheidungen bei der Aufgabenverteilung vermieden werden. Bei einem geschätzten Volumen von 150 Tickets pro Monat amortisieren sich die Entwicklungskosten unter Berücksichtigung der eingesparten Arbeitszeit bereits nach etwa vier Betriebsmonaten. Folglich stellt das Projekt eine nachhaltige Investition zur Effizienzsteigerung der IT Abteilung dar.

6.1 Vergleich von Soll und Ist bei der Zeitplanung

Die IHK Vorgabe von 80 Stunden bildete den verbindlichen Rahmen für die Umsetzung des Projekts TicketsPlease.

Phase	Geplant (h)	Tatsächlich (h)	Abweichung
Analyse & Planung	12	11	-1 h
Entwurf	15	15	
Implementierung	30	31	+1 h
Qualitätssicherung	13	13	
Dokumentation	10	10	
Gesamt	**80 h**	**80 h**	

Tab.6.1: Gegenüberstellung der geplanten und tatsächlich aufgewendeten Projektstunden.

Begründung der Abweichungen:

- **Analyse:** Der Einsatz des Service-Repository-Musters vereinfachte die Architekturentscheidungen und sparte Zeit ein.
- **Implementierung:** Die nahtlose Integration von Tailwind CSS in den ASP.NET Core Buildprozess erforderte unvorhergesehenen Rechercheaufwand.
- **Qualitätssicherung:** Die Implementierung automatisierter Architekturtests mit NetArchTest reduzierte den Bedarf an manuellen Prüfungen erheblich.

6.2 Nachkalkulation der Projektkosten

Die Entwicklungskosten basieren auf dem internen Verrechnungssatz eines Umschülers von 9,00 Euro pro Stunde. Da die geplante Zeitvorgabe von exakt 80 Stunden eingehalten wurde, entsprechen die tatsächlichen Kosten den kalkulierten Plankosten.

$$\text{Personalkosten} = \text{Arbeitsstunden} \times \text{Stundensatz}\, 80\,\text{h} \times 9,00\,€/\text{h} = 720,00\,€$$

$$\text{Gesamtkosten} = \text{Personalkosten} + \text{Sachmittel}\, 720,00\,€ + 200,00\,€ = 920,00\,€$$

6.3 Amortisationsrechnung

Die Wirtschaftlichkeit ergibt sich durch eine signifikante Effizienzsteigerung gegenüber der bisherigen unstrukturierten Aufgabenverwaltung.

Ausgangslage vor Systemeinführung:

$$\text{Arbeitskosten}_{\text{alt}} = \text{Ticketaufkommen} \times \frac{\text{Zeitaufwand}_{\text{alt}}}{60} \times \text{Stundensatz}$$

$$3.000 \times \frac{10\,\text{min}}{60} \times 40,00\,€ = 20.000,00\,€/\text{Jahr}$$

Die jährliche Ersparnis führt zu folgender Amortisationsdauer der Investitionskosten (920,00 €):

$$\text{Gesamtkosten}_{\text{neu}} = \text{Arbeitskosten}_{\text{neu}} + \text{Betriebskosten}$$

$$6.000,00\,€ + 600,00\,€ = 6.600,00\,€/\text{Jahr}$$

Berechnung der Amortisation:

$$\Delta\text{Einsparung} = \text{Kosten}_{\text{alt}} - \text{Kosten}_{\text{neu}}$$

$$20.000,00\,€ - 6.600,00\,€ = 13.400,00\,€/\text{Jahr}$$

$$\text{Amortisationsdauer} = \frac{\text{Investitionskosten}}{\Delta\text{Einsparung}}$$

$$\frac{920,00\,€}{13.400,00\,€} \approx 0,06\,\text{Jahre}$$

Qualitative Bewertung: Das Projekt amortisiert sich betriebswirtschaftlich nach weniger als einem Monat. Ab diesem Zeitpunkt erzielt das Unternehmen durch die Zeitersparnis reine Gewinne. Zusätzlich profitieren alle Abteilungen von einer fehlerfreien Kommunikation, einer transparenten Projektsteuerung und einer nachvollziehbaren Historie für Audits.

7. Fazit & Ausblick

7.1 Zusammenfassung

Das Projekt „TicketsPlease“ wurde erfolgreich und exakt innerhalb des geplanten Zeitrahmens von 80 Stunden realisiert. Sämtliche im Pflichtenheft definierten Anforderungen – von der Ticketverwaltung über das Dashboard bis hin zu Echtzeitbenachrichtigungen – wurden vollständig implementiert. Das Ergebnis ist eine skalierbare, performante ASP.NET Core MVC Webanwendung, die durch den serverseitigen Render-Ansatz und die Integration von Tailwind CSS kurze Ladezeiten und eine hohe Benutzerfreundlichkeit bietet. Besonders hervorzuheben ist die konsequente Umsetzung der Clean Architecture in Kombination mit dem Service-Repository-Pattern. Dies führte zu einer starken Entkopplung der Systemkomponenten, wodurch die Wartbarkeit und zukünftige Erweiterbarkeit der Software maximal gesichert sind.

7.2 Lessons Learned

Die Entwicklung einer Enterprise Anwendung unter Einsatz neuester .NET 10 Features und Architekturmuster erforderte kontinuierliche Evaluation technischer Entscheidungen. Die zentralen Erkenntnisse umfassen:

Architekturdisziplin (Clean Architecture): Die strikte Trennung von Domain, Application, Infrastructure und Web Schicht bedeutete initial einen höheren Planungsaufwand. Bei der späteren Implementierung komplexer Features (wie der Kommentarfunktion oder SignalR) zahlte sich diese Trennung massiv aus, da Nebeneffekte ausgeschlossen wurden.

Service-Repository-Pattern: Die Kapselung von Anwendungslogik in isolierte Services reduzierte die Komplexität der Controller auf ein Minimum. Die Logik ist stark fokussiert, was das Unit Testing der Services erheblich vereinfachte.

Testing und Architekturvalidierung: Die Einführung von NetArchTest verhinderte strukturelle Fehler automatisch im Build Prozess. Der Einsatz automatisierter Komponententests deckte Logikfehler frühzeitig auf.

Modernes UI Tooling (.NET und Tailwind v4.2.2): Die native Integration des Tailwind CLI in den MSBuild Prozess eliminierte die Notwendigkeit einer komplexen Node.js Laufzeitumgebung auf dem Produktionsserver. Das resultierende Zero Dependency Deployment vereinfacht die Wartung der Infrastruktur drastisch.

Professioneller Git Workflow: Die konsequente Nutzung von Feature Branches, kombiniert mit Pull Requests und automatisierten CI Checks via GitHub Actions, garantierte einen stabilen Hauptentwicklungszweig. Atomare Commits erleichterten das Debugging.

7.3 Ausblick

Die modulare Basis der Anwendung erlaubt die nahtlose Integration zukünftiger Erweiterungen. Folgende Features sind für nachfolgende Iterationen geplant:

- **Modernes Identitätsmanagement:** Implementierung von passwortlosen Authentifizierungsverfahren (WebAuthn/FIDO2) mittels Hardware-Token oder Biometrie zur Erhöhung der Kontosicherheit.
- **Vollständiges SLA-Monitoring:** Finalisierung der bereits im Ansatz implementierten Eskalationsstufen durch einen automatisierten Hintergrunddienst zur Überwachung kritischer Lösungszeiten.
- **Performance-Analytik für Teamleiter:** Bereitstellung dedizierter Dashboards mit Metriken zur Team-Effizienz (z. B. MTTR) zur Optimierung interner Workflows.
- **Git-Integration:** Anbindung an Versionsverwaltungssysteme zur direkten Verknüpfung von Entwicklungsaufgaben mit den entsprechenden Quellcode-Änderungen.
- **KI-gestützte Klassifizierung:** Automatisierte Priorisierung eingehender Tickets auf Basis historischer Daten mittels Machine Learning.
-

7.4 Projektabnahme und Übergabe

Die offizielle Projektabnahme erfolgte am 16.04.2026 durch den Projektsponsor und die IT Leitung. Die Abnahme basierte auf dem vorab definierten Testprotokoll sowie dem Soll Ist Vergleich der User Stories. In einer Live Demonstration wurden die Kernprozesse – Anlage, Bearbeitung und Abschluss eines Tickets – inklusive der reaktiven Echtzeitupdates durch SignalR erfolgreich verifiziert.

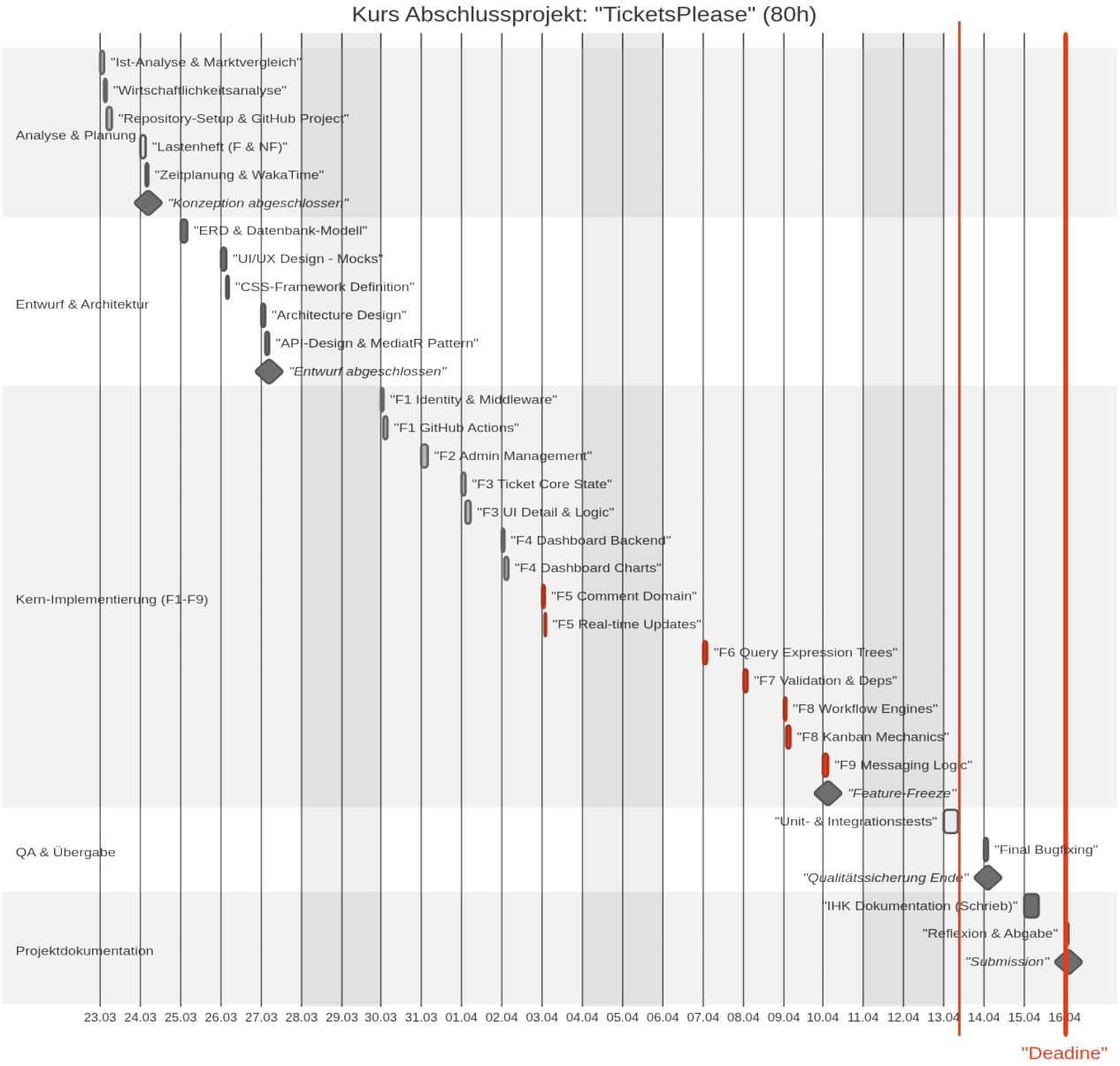
Sämtliche Testfälle wurden fehlerfrei durchlaufen. Die Freigabe für den produktiven Einsatz der Version 1.0 wurde erteilt. Die Übergabe des Quellcodes, der Datenbankmigrationsskripte sowie der Administratoren- und Benutzerdokumentation erfolgte vollumfänglich in digitaler Form

A. Anhang

A.0.1 Detaillierte Aufschlüsselung der Projektphasen in Stunden

Phase	Tätigkeit	Zeit (h)	Startdatum
1. Analyse & Planung		**12 h**	**23.03.2026**
	Ist-Analyse & Marktvergleich	2 h	23.03.2026
	Wirtschaftlichkeitsanalyse (ROI / Make-or-Buy)	2 h	23.03.2026
	Repository-Setup & GitHub Project (Kanban)	3 h	23.03.2026
	Lastenheft: Funktional vs. Nicht-funktional	3 h	24.03.2026
	Zeit- & Ressourcenplanung (Gantt, WakaTime Setup)	2 h	24.03.2026
2. Entwurf		**14 h**	**25.03.2026**
	Datenbankmodellierung (ERD, Relationen)	4 h	25.03.2026
	UI/UX Design: Wireframes & Corporate Skin	3 h	26.03.2026
	IDE-Konfiguration & CI/CD Workflow-Definition	2 h	26.03.2026
	Architecture Design (Clean Arch Layering)	2 h	27.03.2026
	API-Design & MediatR Pattern Definition	3 h	27.03.2026
3. Implementierung (F1-F9)		**34 h**	**30.03.2026**
	F1: SDK Setup & Identity (Auth Middleware)	2 h	30.03.2026
	F1: Shared Projects, GitHub Actions & Env-Setup	2 h	30.03.2026
	F2: Admin: CRUD Logik (Projekte & Benutzer)	4 h	31.03.2026
	F3: Ticket-Core: State Machine & Aggregates	3 h	01.04.2026
	F3: Ticket-Detailview & Edit-Logik	3 h	01.04.2026
	F4: Dashboard: SQL-Aggregation & View-Components	2 h	02.04.2026
	F4: Dashboard: UI-Charts Integration	2 h	02.04.2026
	F5: Kommentare: Domain Events & Persistence	2 h	03.04.2026
	F5: Kommentare: Real-time UI Updates	1 h	03.04.2026
	F6: Filter: Expression Trees & Query Extensions	3 h	07.04.2026
	F7: Abhängigkeiten: Validation Logik & UI	3 h	08.04.2026
	F8: Workflow: Status-Transition Guards	2 h	09.04.2026
	F8: Kanban-Drag&Drop Integration	2 h	09.04.2026
	F9: Messaging: Entity Design & DB-Repository	3 h	10.04.2026
4. Qualitätssicherung		**10 h**	**13.04.2026**
	Unit-Testing: Domain Logic & Commands	4 h	13.04.2026
	Integrationstests: SQL & Repositories	4 h	14.04.2026
	Finales Bugfixing & Dokumentations-Cleanup	2 h	14.04.2026
5. Dokumentation		**10 h**	**15.04.2026**
	IHK Projektdokumentation (Endredaktion)	8 h	15.04.2026
	Fazit, Reflexion & Abgabe	2 h	16.04.2026
Gesamt		**80 h**	

A.0.2 Visuelle Darstellung des Projektverlaufs als Gantt Diagramm



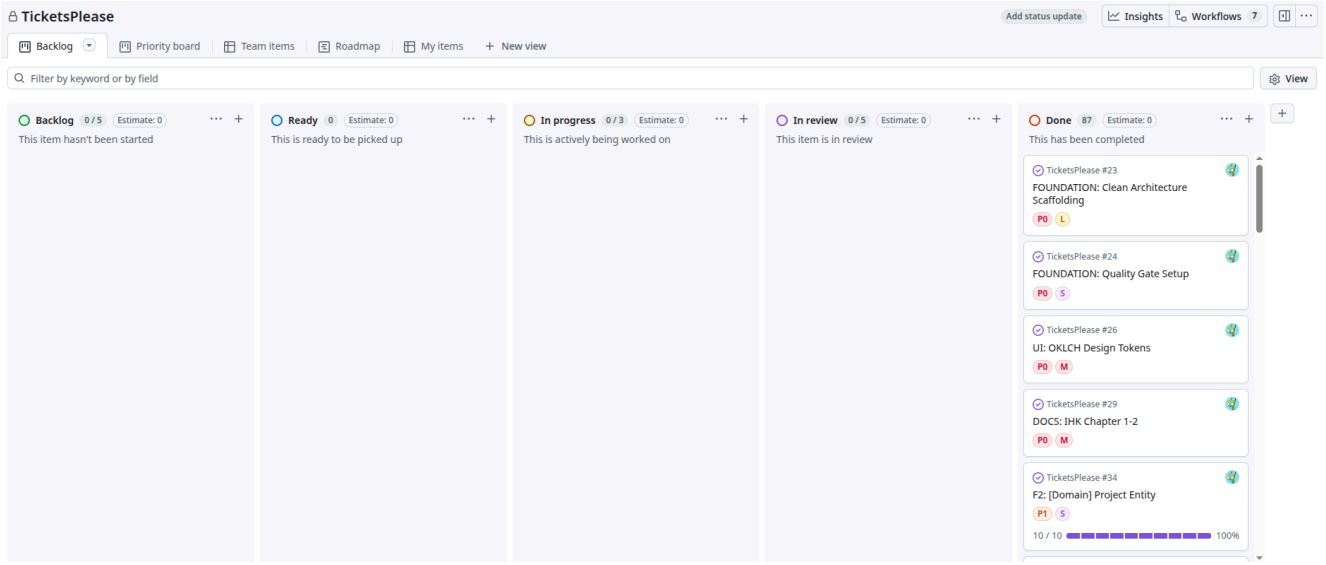


Abb.2.2 Github Kanban Board

A.0 Quellen- und Literaturverzeichnis

Gesetzliche Grundlagen & Normen

- **Bundesministerium der Justiz.** *Gesetz zur Stärkung der Barrierefreiheit (Barrierefreiheitsstärkungsgesetz – BFSG)*. [Online] 2021. https://www.gesetze-im-internet.de/bfsg/ (Abgerufen am: 14.04.2026).
- **Interaktionsrat.** *Web Content Accessibility Guidelines (WCAG) 2.2*. [Online] 2023. https://www.w3.org/TR/WCAG22/ (Abgerufen am: 14.04.2026).
- **Europäisches Parlament.** *Verordnung (EU) 2016/679 (Datenschutz-Grundverordnung)*. [Online] 2016. https://gdpr-info.eu/ (Abgerufen am: 14.04.2026).

Software, Frameworks & Tools

- **Ben-Hamo, Ben.** *NetArchTest.eNet: A fluent unit test library for enforcing architectural rules*. [Software] GitHub, 2024. https://github.com/BenHamo/NetArchTest (Abgerufen am: 14.04.2026).
- **GitHub Inc.** *CodeQL: Semantic code analysis engine*. [Software] 2026. https://codeql.github.com/ (Abgerufen am: 14.04.2026).
- **JetBrains s.r.o.** *Qodana: The code quality platform for CI/CD*. [Software] 2026. https://www.jetbrains.com/qodana/ (Abgerufen am: 14.04.2026).
- **Microsoft Corporation.** *ASP.NET Core Documentation: Overview of ASP.NET Core MVC*. [Online] 2026. https://learn.microsoft.com/en-us/aspnet/core/mvc/overview (Abgerufen am: 14.04.2026).
- **Tailwind Labs Inc.** *Tailwind CSS v4.0 Documentation*. [Online] 2025. https://tailwindcss.com/docs (Abgerufen am: 14.04.2026).
- **WakaTime.** *WakaTime: Dashboards for developers*. [Software] 2026. https://wakatime.com/ (Abgerufen am: 14.04.2026).

Dokumentation & Methodik

- **Fowler, Martin.** *CQRS (Command Query Responsibility Segregation)*. [Online] 2011. https://martinfowler.com/bliki/CQRS.html (Abgerufen am: 14.04.2026).
- **Microsoft Corporation.** *C# Documentation: XML documentation comments*. [Online] 2026. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/xmldoc/ (Abgerufen am: 14.04.2026).

A.0.1 Bildquellen

- **Beispiel GmbH & BitLC:** Logos und Firmenidentität (Bereitgestellt durch den Ausbildungsbetrieb). [vgl.]
- **Eigene Darstellungen:** Alle Diagramme (UML, Architektur, Gantt) und Mockups wurden vom Verfasser selbst, mittels Mermaid.JS[1] und Google Nano Banana erstellt.

[1] [9] Mermaid Dokumentation

A.1 Verzeichnisstruktur

A.2. Quellcode und Dateien

Der vollständige und kommentierte Quellcode aller Projektdateien, Grafiken, Protokolle, Lasten-. und Pflichtenheft et cetera, befindet sich im beiliegenden digitalen Anhang (ZIP-Archiv).

A.3. Diagramme

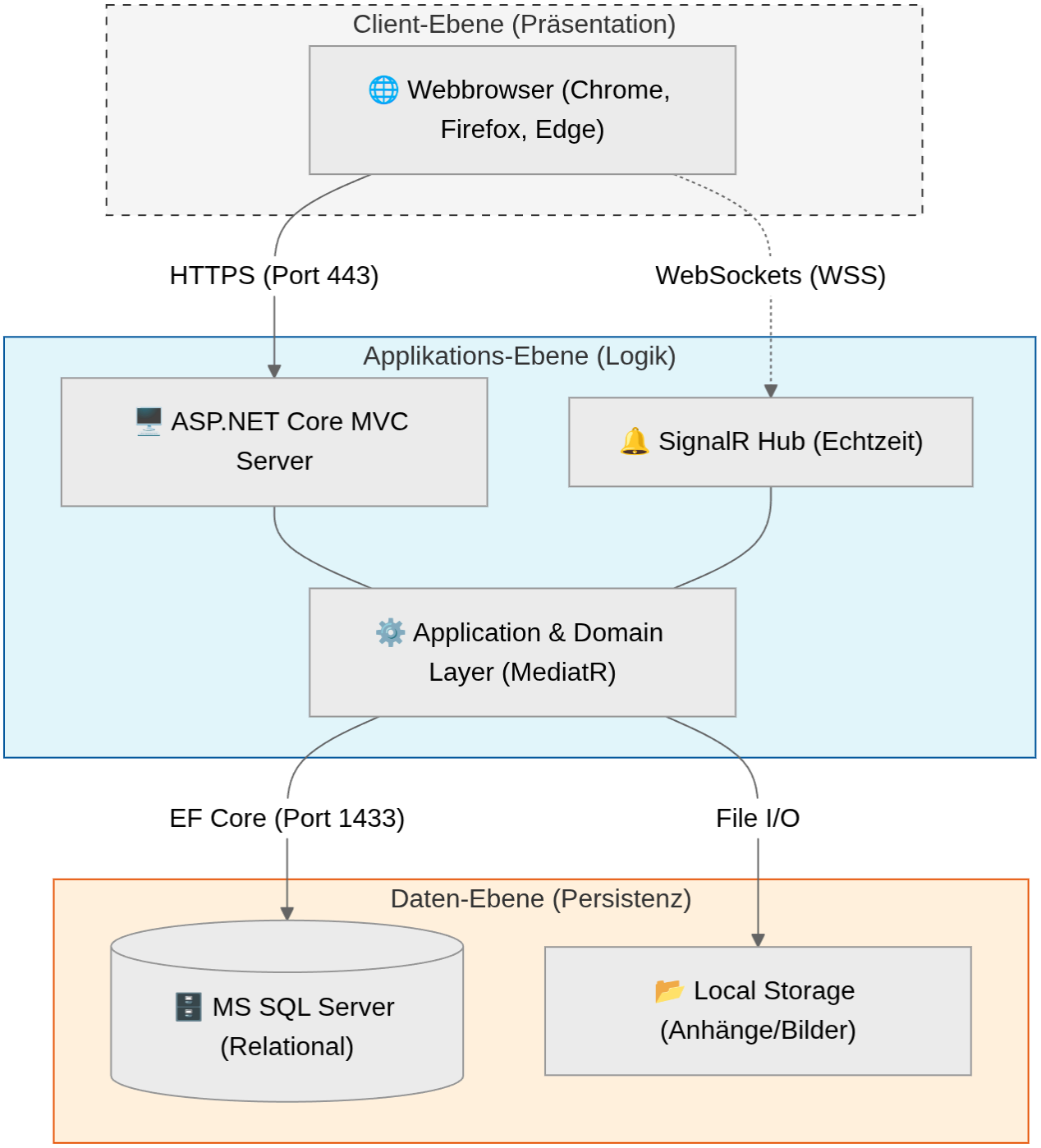
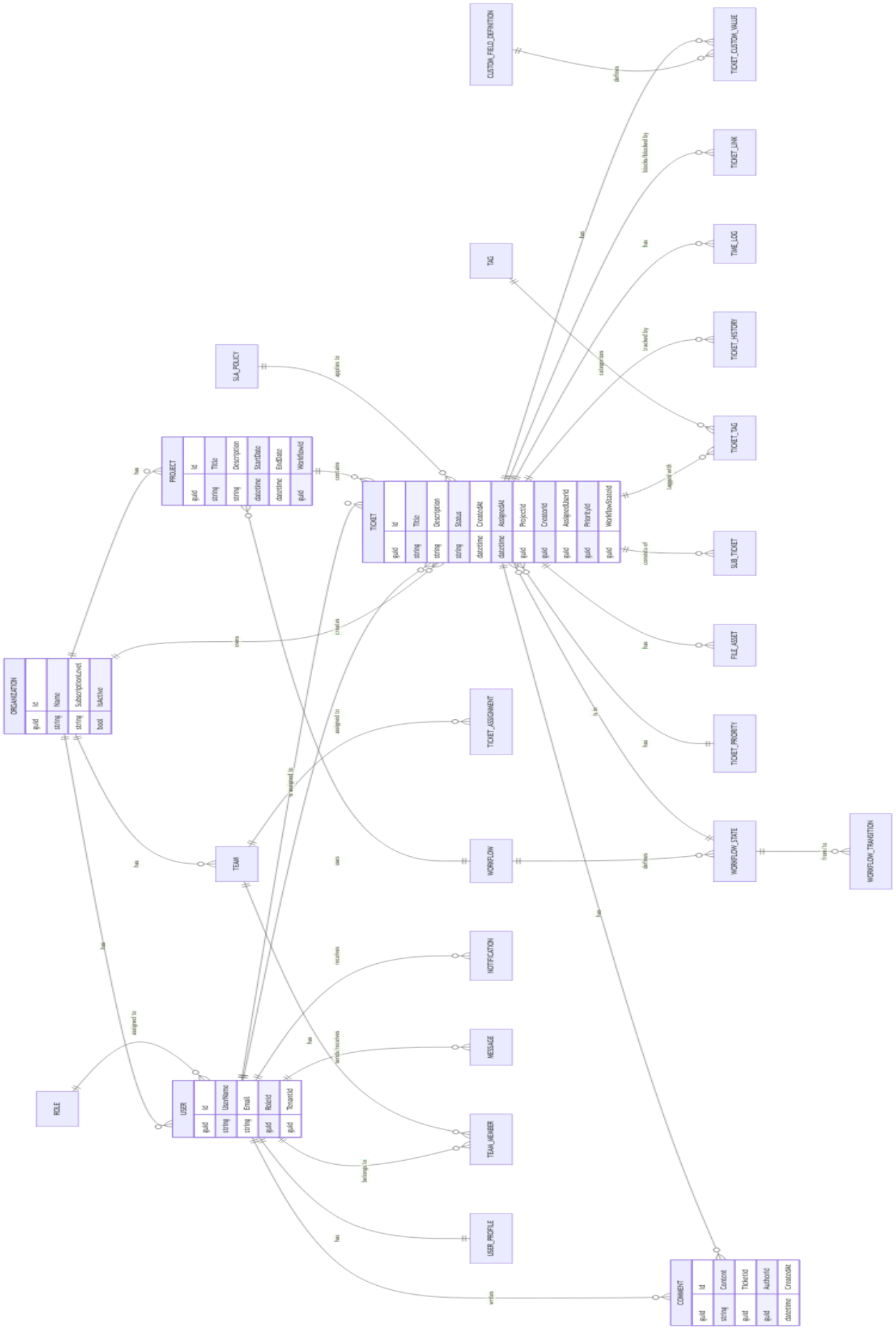


Abb.3.2 3.2 Architekturentwurf


ORGANIZATION
Id guid
Name string
SubscriptionLevel string
IsActive bool
owns
has
PROJECT
Id guid
Title string
Description string
StartDate datetime
EndDate datetime
WorkflowId guid
contains
applies to
SLA_POLICY
TICKET
Id guid
Title string
Description string
Status string
CreatedAt datetime
AssignedAt datetime
ProjectId guid
CreatorId guid
AssignedUserId guid
PriorityId guid
WorkflowStateId guid
creates
assigned to
is assigned to
uses
CUSTOM_FIELD_DEFINITION
defines
TICKET_CUSTOM_VALUE
blocks/blocked by
TICKET_LINK
has
TIME_LOG
tracked by
TICKET_HISTORY
TAG
categorizes
TICKET_TAG
tagged with
SUB_TICKET
consists of
FILE_ASSET
TICKET_PRIORITY
TICKET_ASSIGNMENT
WORKFLOW
WORKFLOW_STATE
defines
from / to
WORKFLOW_TRANSITION
TEAM
NOTIFICATION
receives
MESSAGE
sends/receives
TEAM_MEMBER
belongs to
USER_PROFILE
ROLE
assigned to
USER
Id guid
UserName string
Email string
RoleId guid
TenantId guid
COMMENT
Id guid
Content string
TicketId guid
AuthorId guid
CreatedAt datetime
writes

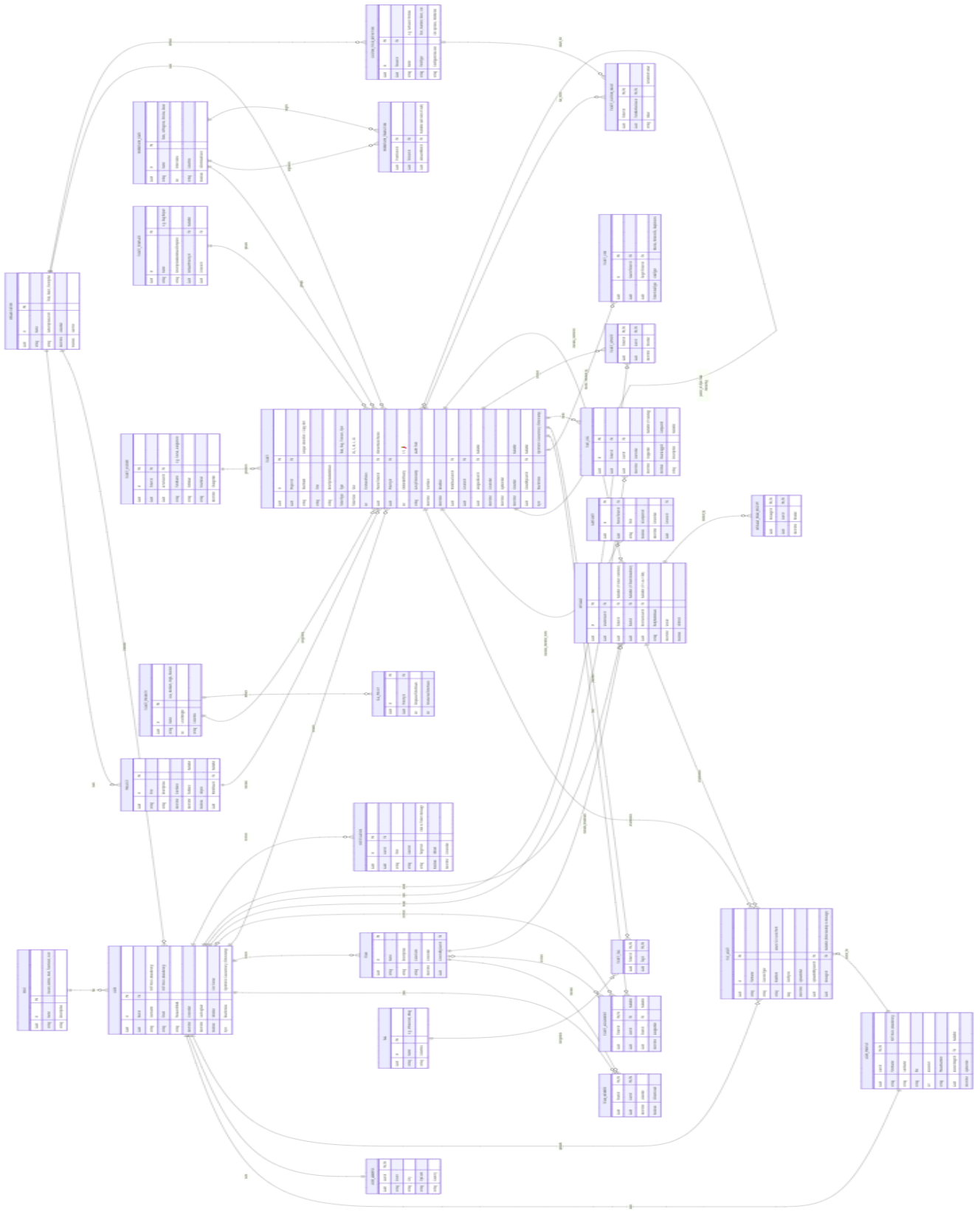


Abb. 3.4 vollständiges Datenmodell, das alle Relationen, Kardinalitäten (1:n, n:m) und Primär-/Fremdschlüssel zeigt.

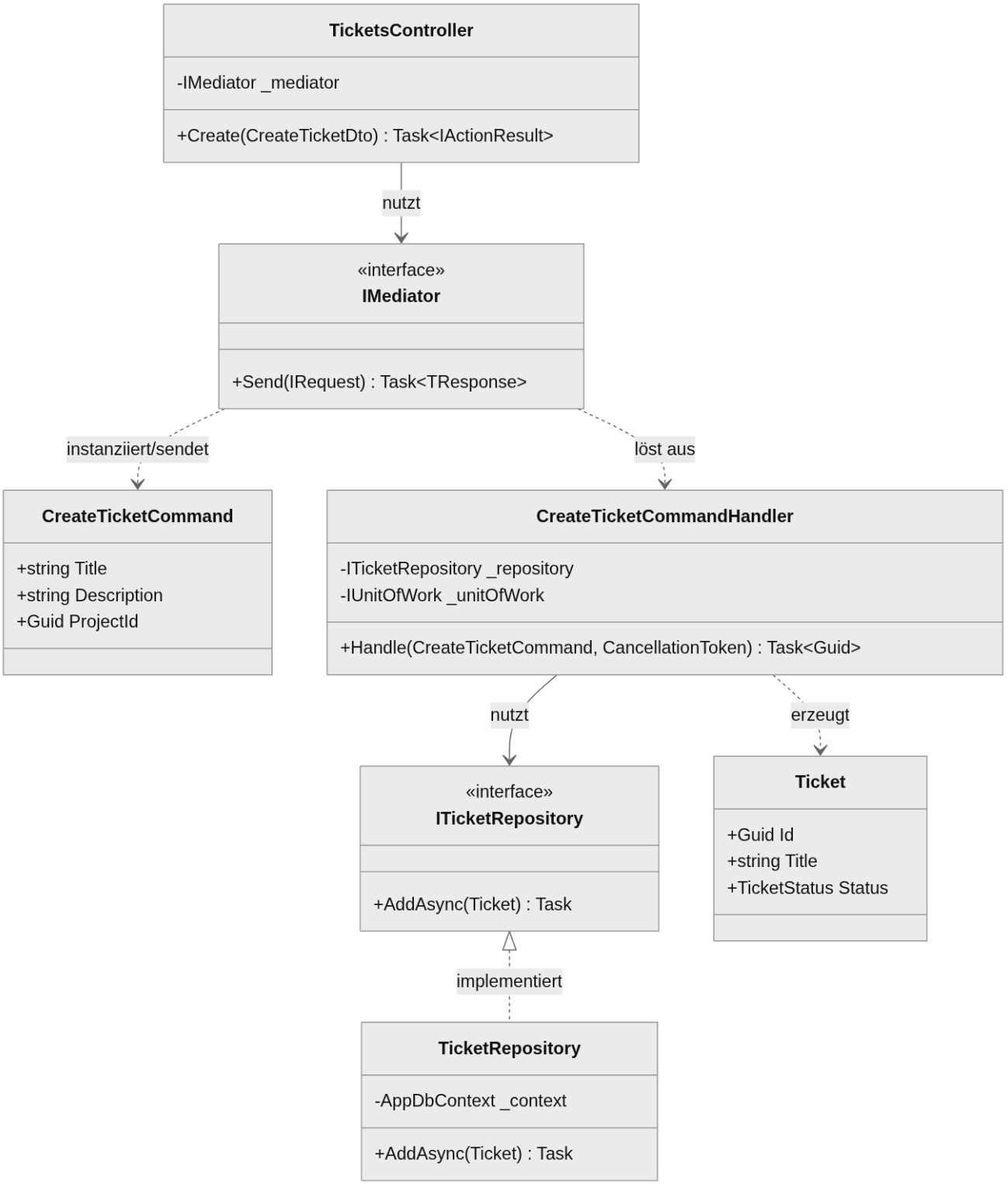


Abb. 3.5 Das Diagramm zeigt den Fluss eines Commands

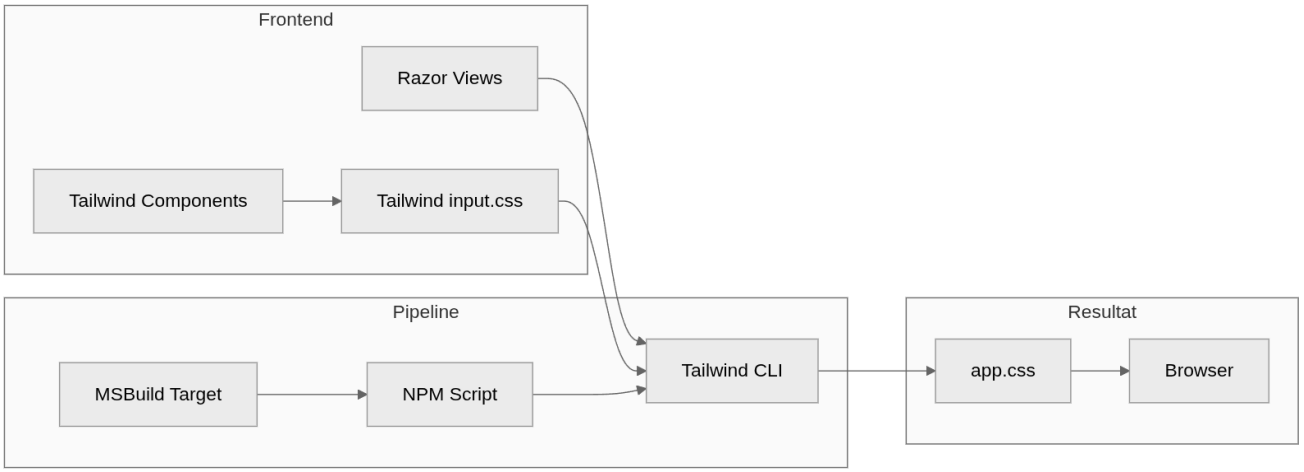


Abb.4.2.3 CSS Architektur und UI Strategie

A.4 Userstories und ADR

Für die initialen Userstories siehe Userstories.PDF

MVP-0001. ASP.NET Core MVC als Web-Framework

Datum: 2026-03-23

Status: Accepted

Kontext

Die Aufgabe (F1.1) fordert eine Web-Anwendung, die mit ASP.NET Core ab Version 8 umgesetzt wird und dem MVC-Muster folgt. Ein einheitliches Design soll durch ein CSS-Framework gewährleistet werden.

Wir müssen entscheiden, welche Variante des ASP.NET Core Frameworks (MVC, Razor Pages, Blazor, Minimal API) für das Ticketsystem eingesetzt wird.

Entscheidung

Wir verwenden **ASP.NET Core 10 MVC** (Model-View-Controller) als primäres Web-Framework mit Razor Views für die serverseitige HTML-Generierung.

Konsequenzen

Positiv

- Exakt die vom Prüfer geforderte MVC-Architektur.
- Serverseitiges Rendering: Kein JavaScript-Framework (React/Vue) nötig.
- Bewährtes Pattern mit klarer Trennung (Controller → View → Model).
- Perfekte Integration mit ASP.NET Core Identity für Auth (F1.3).

Negativ

- Kein SPA-Erlebnis (Single Page Application) – jeder Seitenaufruf ist ein Full-Page-Reload (für MVP akzeptabel).
- Interaktive Features (Drag & Drop) erfordern zusätzliches JavaScript.

Neutral

- Tailwind CSS 4.2.2 wird als CSS-Framework eingesetzt (siehe ADR-0040).

Alternativen

Alternative	Pro	Contra	Entscheidung
ASP.NET MVC	konform, bewährt	Full-Page-Reloads	✅ Gewählt
Razor Pages	Einfacher für CRUD-Views	Weniger Kontrolle, kein MVC	❌ Abgelehnt
Blazor Server	SPA-Feeling	Nicht gefordert, Komplexität	❌ Abgelehnt
Minimal API	Leichtgewichtig	Keine Views, nur API	❌ Abgelehnt

MVP-0002. EF Core Code-First mit SQL Server

Datum: 2026-03-23

Status: Accepted

Kontext

Die Aufgabe (F1.2) verlangt eine SQL Server-Datenbank, angebunden über Entity Framework mit dem Code-First-Ansatz. Offene Migrationen sollen beim App-Start automatisch ausgeführt werden.

Entscheidung

Wir verwenden **Entity Framework Core 10** im **Code-First-Modus** mit Microsoft SQL Server. Migrationen werden beim App-Start automatisch über `context.Database.Migrate()` in `Program.cs` angewendet.

Konsequenzen

Positiv

- Domain-Entities steuern das Datenbankschema (DDD-konform).
- Automatische Migrationen vereinfachen den Entwicklungsprozess.
- Versionierung des Schemas über Git (Migrations-Dateien).
- `localdb` oder Docker SQL Server für lokale Entwicklung.

Negativ

- Automatische Migration bei App-Start kann in Produktion riskant sein (für MVP akzeptabel, für Enterprise separate Strategie nötig).
- Performance-Overhead durch ORM gegenüber Raw SQL (vernachlässigbar).

Neutral

- Connection String wird über `appsettings.Development.json` und `dotnet user-secrets` verwaltet (siehe ADR-0110).

Alternativen

Alternative	Pro	Contra	Entscheidung
EF Core Code-First	konform, DDD	ORM-Overhead	✅ Gewählt
EF Core DB-First	Bestehende DB nutzbar	Kein Code-First, aufwändiger	❌ Abgelehnt
Dapper (Micro-ORM)	Performanter	Kein EF, kein Code-First	❌ Abgelehnt
ADO.NET Raw	Volle Kontrolle	Viel Boilerplate, unsicher	❌ Abgelehnt

MVP-0003. ASP.NET Core Identity für Authentifizierung

Datum: 2026-03-23

Status: Accepted

Kontext

Die Aufgabe (F1.3) fordert eine Authentifizierung/Autorisierung mit dem Identity-Framework. Es müssen mindestens die Rollen Admin, Developer und Tester existieren. Es wird ein eigener AccountController benötigt. Nicht angemeldete Benutzer sehen nur die Startseite.

Entscheidung

Wir verwenden **ASP.NET Core Identity** mit Cookie-basierter Authentifizierung. Die Implementierung umfasst:

1. **Eigener `AccountController`** mit Login/Logout-Actions (kein Scaffolding der Identity-UI).
2. **Standard-Rollen:** Admin, Teamlead, User, ProductOwner, Stakeholder – geseeded über DbInitialiser.
3. **`[Authorize]`-Attribut** global auf alle Controller außer der Startseite.
4. **Rollenbasierte Autorisierung** via `[Authorize(Roles = "Admin")]`.

Konsequenzen

Positiv

- Exakt konform: Identity-Framework, eigener AccountController.
- Cookie-Auth ist einfach und sicher für MVC-Anwendungen.
- Role-Based Access Control (RBAC) sofort nutzbar.
- Seed-Daten garantieren mindestens je einen User pro Rolle.

Negativ

- Kein JWT/Bearer Token Support (für MVP nicht nötig, für API in Enterprise nachrüstbar).
- Benutzerverwaltung (F2.3) ist komplex, da nur über `UserManager` möglich (kein Scaffolding).

Neutral

- Passwörter werden automatisch via Identity gehasht (Pbkdf2).
- `ApplicationUser : IdentityUser` erweitert den Standard-User.

Alternativen

Alternative	Pro	Contra	Entscheidung
ASP.NET Identity	Pflicht, integriert	Etwas komplex	✅ Gewählt
Eigene Auth-Lösung	Volle Kontrolle	Unsicher, viel Aufwand	❌ Abgelehnt
Keycloak / IdentityServer	Enterprise-Grade	Overkill für MVP	❌ Abgelehnt

MVP-0004. Project-Entity und Admin-CRUD

Datum: 2026-03-23

Status: Accepted

Kontext

Die Aufgabe (F2.1, F2.2) fordert einen eigenen Admin-Bereich mit CRUD-Funktionalität für Projekte. Ein Projekt besteht aus Titel, Beschreibung, Startdatum (Pflicht) und Enddatum (optional). Nur Admins dürfen Projekte verwalten. Tickets werden später einem Projekt zugeordnet.

Das `Project`-Entity existiert aktuell **nicht** im Domain Layer und muss neu erstellt werden.

Entscheidung

1. **Neues `Project`-Entity** im Domain Layer mit den Properties:
   - `Title` (string, Pflicht, max. 200 Zeichen)
   - `Description` (string, Pflicht)
   - `StartDate` (DateTimeOffset, Pflicht)
   - `EndDate` (DateTimeOffset?, Optional)
   - Erbt von `BaseEntity` (Id, TenantId, IsDeleted, RowVersion)
2. **Admin-Bereich** unter `/Admin/` mit eigenem Area oder Controller-Prefix.
3. **Autorisierung:** `[Authorize(Roles = "Admin")]` auf dem gesamten Admin-Bereich.
4. **Zentraler Service-Layer:** Integration über IProjectService für alle CRUD Operationen.
5. **Scaffold-Unterstützung:** Razor Views für Create/Edit/Delete/Index.

Konsequenzen

Positiv

- Saubere Domain-Entity mit Rich-Model-Pattern.
- Projekte sind von Tag 1 an Aggregate Roots (Tickets referenzieren sie).
- Admin-Bereich ist rollenbasiert abgesichert.
- Anforderung F2.2 vollständig abgedeckt.

Negativ

- Neue EF Core Migration erforderlich für die `Projects`-Tabelle.
- Admin-Bereich erfordert separate Views und Navigation.

Neutral

- Ein Projekt kann als „beendet" gelten, wenn `EndDate < DateTime.Now`.
- Nur offene Projekte (nicht beendet/gelöscht) können Tickets zugeordnet werden (F3.1 Anforderung).

Alternativen

Alternative	Pro	Contra	Entscheidung
Eigene Project-Entity	DDD-konform, flexibel	Migration nötig	✅ Gewählt
Projekte als Enum/Liste	Einfacher	Kein CRUD, nicht erweiterbar	❌ Abgelehnt
Projekte als Tags	Flexible Zuordnung	Kein Admin-CRUD möglich	❌ Abgelehnt

MVP-0005. Ticket-Lifecycle (Erstellen, Bearbeiten, Schließen)

Datum: 2026-03-23

Status: Accepted

Kontext

Die Aufgabe (F3.1–F3.4) definiert den vollständigen Ticket-Lifecycle: Erstellen, Auflisten, Detailansicht, Bearbeiten und Schließen. Strikte Berechtigungsregeln bestimmen, wer welche Aktionen ausführen darf.

Entscheidung

Das `Ticket`-Entity im Domain Layer wird mit folgenden MVP-Properties implementiert:

Properties (Rich Domain Model)

- `Title` (string, Pflicht), `Description` (string, Pflicht)
- `ProjectId` (Guid, FK → Project, Pflicht, nur offene Projekte)
- `CreatorId` (Guid, FK → User, automatisch = angemeldeter Benutzer)
- `CreatedAt` (DateTimeOffset, automatisch)
- `AssignedUserId` (Guid?, nullable)
- `AssignedAt` (DateTimeOffset?, automatisch bei Zuweisung)
- `ClosedByUserId` (Guid?, F3.4), `ClosedAt` (DateTimeOffset?, F3.4)

Berechtigungslogik (Domain-Methoden)

- **Bearbeiten** (`ticket.Update(userId)`): Nur Admin, Ersteller oder Zugewiesener. Änderbar: Beschreibung, AssignedUserId.
- **Schließen** (`ticket.Close(userId)`): Nur Admin, Ersteller oder Zugewiesener. Setzt `ClosedByUserId` und `ClosedAt`.
- **Sortierung:** Ticket-Liste nach Projekt + Erstellungsdatum (absteigend).

Konsequenzen

Positiv

- Rich Domain Model: Geschäftslogik liegt im Entity, nicht im Controller.
- Klare Berechtigungsprüfung via Domain-Methoden.
- Alle Akzeptanzkriterien (F3.1–F3.4) vollständig abgedeckt.

Negativ

- `private set` auf allen Properties erfordert Verhaltensmethoden.
- Domain-Validierung muss sowohl im Entity als auch in FluentValidation gespiegelt werden.

Alternativen

Alternative	Pro	Contra	Entscheidung
Rich Domain Model	DDD, testbar	Mehr Code im Entity	✅ Gewählt
Anemic Model + Service	Einfacher	Verletzt DDD, weniger testbar	❌ Abgelehnt

MVP-0006. Startseite mit Statistik-Aggregationen

Datum: 2026-03-23

Status: Accepted

Kontext

Die Aufgabe (F4.1, F4.2) fordert eine zentrale Startseite, die auch ohne Login erreichbar ist. Sie soll Statistiken anzeigen: Tickets (Gesamt/offen/geschlossen), Projekte (Gesamt/offen/beendet) und optional Benutzer (Gesamt/pro Rolle).

Entscheidung20.00

1. **Startseite** unter Root-URL (`/`), erreichbar über `HomeController`.
2. `[AllowAnonymous]` auf der Index-Action, alle anderen Bereiche erfordern Login.
3. **Statistik-Berechnung** via IDashboardService: Liefert ein DashboardStatisticsDto mit:
    - TotalTickets, OpenTickets, ClosedTickets
    - TotalProjects, ActiveProjects, CompletedProjects
    - (Optional) TotalUsers, UsersByRole Dictionary
4. **EF Core Aggregation** via `CountAsync()` mit `AsNoTracking()`.
5. **Links/Buttons** zum Ticket- und Admin-Bereich (Admin-Link nur für Admins sichtbar via `User.IsInRole("Admin")`).

Konsequenzen

Positiv

- Klar definiertes Query-Pattern (CQRS).
- Ein einziger DB-Roundtrip für alle Statistiken (Projektion).
- Auch unangemeldete Benutzer sehen Statistiken (Information Radiator).

Negativ

- Statistiken könnten bei vielen Tickets langsam werden (für MVP mit < 1000 Tickets irrelevant; für Enterprise: Redis-Caching).

Alternativen

Alternative	Pro	Contra	Entscheidung
MediatR Query (CQRS)	Sauber, testbar	Etwas mehr Code	✅ Gewählt
ViewComponent direkt	Weniger Code	Kein CQRS, schwer testbar	❌ Abgelehnt
Cached Counter-Tabelle	Performant	Overkill für MVP	❌ Abgelehnt

MVP-0007. Ticket-Kommentarsystem

Datum: 2026-03-23

Status: Accepted

Kontext

Die Aufgabe (F5.1) fordert ein Kommentarsystem für Tickets. Kommentare haben Inhalt, TicketID, Ersteller und Erstellzeitpunkt. Sie werden auf der Ticket-Detailseite angezeigt (neueste zuerst) und können dort angelegt werden.

Entscheidung

Wir nutzen das bestehende **`Message`-Entity** (bereits im Domain Layer) als Kommentar-Objekt. Ein Kommentar ist eine Message mit gesetztem `TicketId`-FK (und `ReceiverUserId = null`, `TeamId = null`).

Implementierung

1. **Keine neue Entity nötig** – Message-Entity deckt Kommentare ab.
2. **Service Integration:** IMessageService übernimmt die Logik zur Speicherung.
3. **Methoden:** AddTicketCommentAsync(ticketId, bodyText) – setzt SenderUserId automatisch auf den angemeldeten User.
4. **UI:** Kommentarliste auf der Ticket-Detailseite als Partial View.

Konsequenzen

Positiv

- Wiederverwendung des bestehenden Message-Entity (DRY).
- Polymorphes Design: Gleiche Entity für Kommentare, DMs und Broadcasts.
- Einfache Query über FK-Filter auf `TicketId`.

Negativ

- Message-Entity enthält Felder (TeamId, ReceiverUserId), die für Kommentare irrelevant sind (nullable, kein Problem).
- Kein Thread/Antwort-System (für MVP nicht gefordert).

Alternativen

Alternative	Pro	Contra	Entscheidung
Message als Kommentar	DRY, bereits vorhanden	Ungenutzte nullable Felder	✅ Gewählt
Eigene Comment-Entity	Spezialisiert	Code-Duplizierung	❌ Abgelehnt

MVP-0008. Ticket-Filterung via LINQ-Queries

Datum: 2026-03-23

Status: Accepted

Kontext

Die Aufgabe (F6.1–F6.3) fordert eine Filterung der Ticket-Liste nach drei Kriterien: Projekt, zugeordneter Benutzer und Ersteller. Die Seite soll bei Filterung neu laden (serverseitiges Filtern).

Entscheidung

Wir implementieren serverseitige Filterung über **Query-String-Parameter** und **EF Core LINQ-Queries** mit dynamischer Komposition.

Implementierung

1. **Query-Parameter:** `?projectId=&assignedUserId=&creatorId=`
2. **ITicketService** nimmt optionale Filter-Parameter in der GetFilteredTicketsAsync Methode entgegen.
3. **LINQ-Komposition:** Filter werden innerhalb des Repositorys nur angehängt, wenn der Parameter gesetzt ist (dynamisches IQueryable<T>.Where()).
4. **UI:** Dropdown-Selects über der Ticket-Liste. Form mit GET-Method, Submit lädt die Seite mit Filtern neu.
5. **`AsNoTracking()`** für alle Lese-Queries (Performance).

```
// Beispiel: Dynamische Filter-Komposition
var query = _context.Tickets.AsNoTracking();
if (projectId.HasValue)
    query = query.Where(t => t.ProjectId == projectId);
if (assignedUserId.HasValue)
    query = query.Where(t => t.AssignedUserId == assignedUserId);
// "Nicht zugeordnet" = AssignedUserId == null
```

Konsequenzen

Positiv

- Einfach, performant, konform.
- Server-seitig: Kein JavaScript nötig.
- Dynamische LINQ-Komposition ist erweiterbar.
- Filter über URL teilbar (Bookmarkable).

Negativ

- Full-Page-Reload bei jeder Filteränderung (für MVP akzeptabel).
- Kein Faceted Search mit Counts (Enterprise Feature).

Alternativen

Alternative	Pro	Contra	Entscheidung
Server-Side LINQ	Einfach, konform	Page-Reload	✅ Gewählt
Client-Side JS Filter	Kein Reload	Alle Daten ins Frontend	❌ Abgelehnt
OData / GraphQL	Flexibel	Overkill für MVP	❌ Abgelehnt

MVP-0009. Ticket-Abhängigkeiten (Blocking)

Datum: 2026-03-23

Status: Accepted

Kontext

Die Aufgabe (F7.1) fordert die Möglichkeit, Tickets als „blockiert durch andere Tickets" zu markieren. Ein Ticket kann nur geschlossen werden, wenn alle blockierenden Tickets bereits geschlossen sind.

Entscheidung

Wir implementieren eine **n:m-Beziehung** (Self-Referencing Many-to-Many) über eine Join-Tabelle `TicketDependency`.

Implementierung

1. **Neue Join-Entity:** `TicketDependency` (oder Auflösungstabelle)
   - `BlockedTicketId` (Guid, FK → Ticket) – das blockierte Ticket
   - `BlockingTicketId` (Guid, FK → Ticket) – das blockierende Ticket
   - Composite PK: (`BlockedTicketId`, `BlockingTicketId`)
2. **Domain-Methode auf Ticket:**
   - `ticket.AddBlocker(blockingTicketId)` – fügt Abhängigkeit hinzu.
   - `ticket.CanClose()` → prüft, ob alle Blocker geschlossen sind.
   - `ticket.Close(userId)` → wirft Exception, wenn `!CanClose()`.
3. **UI auf Detailseite:**
   - Multi-Select Dropdown zum Auswählen blockierender Tickets.
   - Liste der blockierenden Tickets als Links.
   - Close-Button deaktiviert, wenn offene Blocker vorhanden.

Konsequenzen

Positiv

- Saubere relationale Modellierung (keine JSON-Arrays im Ticket).
- Domain-Logik erzwingt Geschäftsregel „Close nur wenn alle Blocker geschlossen".
- Einfache Query: `ticket.Blockers.All(b => b.IsClosed)`.

Negativ

- Neue Entity und Migration erforderlich.
- Zirkuläre Abhängigkeiten sind möglich (A blockiert B, B blockiert A) – für MVP keine Zykluserkennung implementiert.

Alternativen

Alternative	Pro	Contra	Entscheidung
Join-Tabelle (n:m)	Relational, 3NF	Neue Entity/Migration	✅ Gewählt
JSON-Array im Ticket	Kein Join nötig	Nicht relational, no FK	❌ Abgelehnt
Separate Graph-DB	Perfekt für Graphen	Massiv Overkill	❌ Abgelehnt

MVP-0010. Workflow-Engine (Status-Verwaltung)

Datum: 2026-03-23

Status: Accepted

Kontext

Die Aufgabe (F8.1) fordert eine Workflow-Verwaltung für Tickets. Admins können Workflows erstellen (CRUD). Ein Workflow hat eine ID und Bezeichnung und besteht aus einer Reihe von Status. Jedem Projekt wird genau ein Workflow zugeordnet.

Optional: Status-Folge-Regeln (welche Status-Übergänge erlaubt sind) und rollenbasierte Status-Vergabe.

Entscheidung

Wir nutzen das bestehende **`WorkflowState`-Entity** (bereits im Domain Layer) und erweitern es um ein **`Workflow`-Aggregat**.

Implementierung

1. **Neues `Workflow`-Entity** (Aggregat Root):
   - `Id` (Guid), `Name` (string, z.B. „Standard", „Bug-Tracking")
   - `States` (Collection von `WorkflowState`)
2. **Bestehendes `WorkflowState`-Entity** erweitern:
   - `WorkflowId` (Guid, FK → Workflow)
   - `Name`, `OrderIndex`, `ColorHex`, `IsTerminalState`
3. **Project ↔ Workflow:** `Project.WorkflowId` (Guid, FK → Workflow).
4. **Admin-CRUD:** `/Admin/Workflows/` mit Create, Edit, Delete, Index.
5. **Ticket-Status:** Ticket referenziert `WorkflowStateId` statt String-basiertem Status.

Optional (wenn Zeit)

- `WorkflowTransition` (bereits im Domain Layer): Definiert erlaubte Status-Übergänge und Rollen-Berechtigungen pro Übergang.

Konsequenzen

Positiv

- Dynamische Workflows pro Projekt (nicht hardcoded).
- WorkflowState-Entity bereits vorhanden, minimaler Aufwand.
- Admins können projektspezifische Prozesse definieren.
- Grundlage für Enterprise-Kanban-Board (Phase 2).

Negativ

- Komplexer als ein simples String-Feld (`Status = "Offen"`).
- Neue Entities und Migrationen erforderlich.
- Default-Workflow muss geseeded werden.

Alternativen

Alternative	Pro	Contra	Entscheidung
Workflow-Entity (Dynamisch)	Flexibel, konform	Mehr Code, Migration	✅ Gewählt
Status als Enum	Extrem einfach	Nicht dynamisch, kein CRUD	❌ Abgelehnt
Status als Lookup-Table	Einfach	Kein Workflow-Konzept	❌ Abgelehnt

MVP-0011. Nachrichten-System (User-to-User)

Datum: 2026-03-23

Status: Accepted

Kontext

Die Aufgabe (F9.1) fordert ein Nachrichtensystem, über das Benutzer außerhalb von Tickets miteinander kommunizieren können. Nachrichten bestehen aus Absender, Empfänger, Zeitstempel und Textinhalt. Jeder Benutzer bekommt eine Nachrichten-Seite mit einer nach Absender/Empfänger gruppierten Übersicht.

Entscheidung

Wir nutzen das bestehende **`Message`-Entity** (bereits im Domain Layer). Eine Direktnachricht (DM) ist eine Message mit gesetztem `ReceiverUserId` (und `TicketId = null`, `TeamId = null`).

Implementierung

1. **Kein neues Entity nötig** – Message-Entity deckt DMs ab.
2. **`MessagesController`** mit:
   - `Index` – Übersicht aller Konversationen (gruppiert nach Partner).
   - `Conversation(userId)` – Alle Nachrichten mit einem bestimmten User.
   - `Send(receiverId, body)` – Neue Nachricht erstellen.
3. **Gruppierungs-Query:** Messages nach dem jeweils anderen User gruppieren (Sender → Empfänger oder Empfänger → Sender).
4. **SenderUserId** wird automatisch auf den angemeldeten User gesetzt.
5. **SentAt** wird automatisch auf `DateTimeOffset.UtcNow` gesetzt.

Konsequenzen

Positiv

- Wiederverwendung des Message-Entity (DRY, konsistent mit F5).
- Polymorphes Message-Pattern: TicketId→Kommentar, ReceiverUserId→DM.
- Einfache Konversations-Gruppierung via LINQ GroupBy.

Negativ

- Kein Echtzeit-Chat (für MVP ist Full-Page-Reload akzeptabel; SignalR ist Enterprise Phase 2).
- Keine Read-Receipts im MVP (MessageReadReceipt ist Enterprise).
- Rein textbasiert, kein Markdown-Rendering (Enterprise).

Alternativen

Alternative	Pro	Contra	Entscheidung
Message-Entity (DM)	DRY, polymorphes Design	Kein Echtzeit	✅ Gewählt
Eigene DM-Entity	Spezialisiert	Code-Duplizierung	❌ Abgelehnt
SignalR Real-Time Chat	Echtzeit	Overkill für MVP	❌ Abgelehnt

A.5 Code-Beispiele

```
namespace TicketsPlease.Domain.Entities;

/// <summary>
/// Repräsentiert ein Ticket innerhalb des Systems.
/// Diese Klasse enthält die Kern-Geschäftslogik für Ticket-Operationen.
/// </summary>
public class Ticket : BaseAuditableEntity, IAggregateRoot
{
    /// <summary>
    /// Initialisiert eine neue Instanz der Ticket-Klasse.
    /// </summary>
    /// <param name="title">Der Titel des Tickets.</param>
    /// <param name="description">Die detaillierte Beschreibung des Problems oder der
Aufgabe.</param>
    /// <param name="projectId">Die eindeutige Identifikation des zugehörigen Projekts.</param>
    public Ticket(string title, string description, Guid projectId)
    {
        Title = title;
        Description = description;
        ProjectId = projectId;
        CreatedAt = DateTime.UtcNow;
    }
}
```

Listing 4.2.1: Implementierung der Ticket Entität mit XML Dokumentation nach IHK Vorgaben.

```
namespace TicketsPlease.Application.Services;

/// <summary>
/// Implementierung des Ticket-Services zur Steuerung des Kanban-Boards.
/// </summary>
public class TicketService : ITicketService
{
    private readonly ITicketRepository _ticketRepository;

    /// <summary>
    /// Initialisiert den Service mit dem erforderlichen Repository.
    /// </summary>
    /// <param name="ticketRepository">Das Repository für den Zugriff auf Ticket-Daten.</param>
    public TicketService(ITicketRepository ticketRepository)
    {
        _ticketRepository = ticketRepository;
    }

    /// <summary>
    /// Erstellt ein neues Ticket und weist es einem Projekt zu.
    /// </summary>
    /// <param name="dto">Das DTO mit den Ticket-Informationen.</param>
    public async Task CreateTicketAsync(CreateTicketDto dto)
    {
        var ticket = new Ticket(dto.Title, dto.Description, dto.ProjectId);
        await _ticketRepository.AddAsync(ticket);
        await _ticketRepository.SaveChangesAsync();
    }
}
```

Listing 4.2.2: Implementierung der Service-Struktur zur Entkopplung der Anwendungslogik.

```
namespace TicketsPlease.Infrastructure.Persistence;

/// <summary>
/// Stellt Methoden zur Initialisierung der Datenbank bereit.
/// Diese Klasse sorgt für den Aufbau des Datenbestands bei Systemstart.
/// </summary>
public static class DbInitialiser
{
    /// <summary>
    /// VORGABE: Befüllt die Datenbank mit synthetischen Testdaten gemäß ADR 0120.
    /// Dies dient der Einhaltung der Datenschutz Grundverordnung.
    /// </summary>
    /// <param name="context">Der Datenbankkontext für den Zugriff auf die Tabellen.</param>
    /// <returns>Ein Task Objekt für die asynchrone Ausführung.</returns>
    public static async Task SeedSyntheticDataAsync(AppDbContext context)
    {
        // Prüfen ob bereits Benutzer vorhanden sind
        if (context.Users.Any())
        {
            return;
        }

        // Erzeugung fiktiver Benutzerprofile ohne Realdatenbezug
        var testUser = new User
        {
            UserName = "test.developer@example.com",
            Email = "test.developer@example.com",
            EmailConfirmed = true
        };

        await context.Users.AddAsync(testUser);
        await context.SaveChangesAsync();
    }
}
```

Listing.4.3.1 DB Seeder

```
/// <summary>
/// Generiert synthetische Testdaten fuer die Entitaet Ticket zur Einhaltung der strengen Datenschutzvorgaben.
/// Implementiert die Vorgaben zur obligatorischen Nutzung synthetischer Daten in Entwicklungsnetzwerken.
/// </summary>
/// <remarks>
/// Die Klasse nutzt die Bibliothek Bogus zur deterministischen Erzeugung von Pseudodaten.
/// Der Seed wird festgesetzt um reproduzierbare Testergebnisse bei Regressionstests zu garantieren.
/// Dies entspricht den Best Practices fuer Test Driven Development.
/// </remarks>
public static class SyntheticTicketFaker
{
    /// <summary>
    /// Erstellt eine definierte Anzahl an synthetischen Tickets.
    /// Nutzt lokalisierte Faker Profile fuer realistische deutsche Testdaten.
    /// </summary>
    /// <param name="count">Die exakte Anzahl der zu generierenden Datensaetze.</param>
    /// <returns>Eine vollstaendig initialisierte Liste mit synthetischen Ticketinstanzen.</returns>
    /// <exception cref="System.ArgumentOutOfRangeException">Wird geworfen sobald ein negativer Wert uebergeben
wird.</exception>
    public static List<Ticket> GenerateSyntheticTickets(int count)
    {
        if (count <= 0)
        {
            throw new System.ArgumentOutOfRangeException(nameof(count), "Die Anzahl muss strikt positiv sein.");
        }
        Bogus.Randomizer.Seed = new System.Random(1337);
        var ticketFaker = new Bogus.Faker<Ticket>("de")
            .RuleFor(ticket => ticket.Id, faker => System.Guid.NewGuid())
            .RuleFor(ticket => ticket.Title, faker => faker.Commerce.ProductName())
            .RuleFor(ticket => ticket.Description, faker => faker.Lorem.Paragraph())
            .RuleFor(ticket => ticket.CreatedAt, faker => faker.Date.PastOffset())
            .RuleFor(ticket => ticket.CreatedBy, faker => faker.Internet.Email())
            .RuleFor(ticket => ticket.IsActive, faker => true);

        return ticketFaker.Generate(count);
    }
}
```

Listing.3.6 DB Seeding mit Bogus.Faker Daten

Das Codebeispiel veranschaulicht die Umsetzung der Barrierefreiheit in der Präsentationsschicht. Die Lösung verknüpft Lokalisierung und das Corporate Skinning mit semantischen ARIA Attributen unter Verwendung eines dedizierten TagHelpers sowie einer strukturierten Razor View.

```
using Microsoft.AspNetCore.Razor.TagHelpers;
using TicketsPlease.Application.Common.Interfaces;
namespace TicketsPlease.Web.TagHelpers;
/// <summary>
/// Generiert barrierefreie Schaltflaechen gemaess WCAG 2.2 und BFSG 2026.
/// </summary>
/// <remarks>
/// Setzt automatisch relevante ARIA Attribute ein und garantiert ausreichende
/// Farbkontraste durch den dynamischen Abgleich mit dem Corporate Skinning Provider.
/// </remarks>
[HtmlTargetElement("a11y-button")]
public class A11yButtonTagHelper : TagHelper
{
    private readonly ICorporateSkinProvider _skinProvider;

    /// <summary>
    /// Initialisiert eine neue Instanz der A11yButtonTagHelper Klasse.
    /// </summary>
    /// <param name="skinProvider">Der injizierte Service zur Ermittlung des kundenspezifischen Brandings.</param>
    /// <exception cref="System.ArgumentNullException">Wird geworfen, falls der Provider null ist.</exception>
    public A11yButtonTagHelper(ICorporateSkinProvider skinProvider)
    {
        _skinProvider = skinProvider ?? throw new System.ArgumentNullException(nameof(skinProvider));
```

```
    }
    /// <summary>
    /// Legt fest, ob das Element visuell und semantisch als aktiv markiert werden soll.
    /// </summary>
    /// <value>Ein boolescher Wert, der den aria-pressed Status steuert.</value>
    [HtmlAttributeName("is-active")]
    public bool IsActive { get; set; }
    /// <summary>
    /// Der lokalisierte Text fuer Screenreader.
    /// </summary>
    /// <value>Eine Zeichenkette, die als aria-label im DOM gerendert wird.</value>
    [HtmlAttributeName("aria-label")]
    public string AriaLabel { get; set; }
    /// <summary>
    /// Verarbeitet die TagHelper Ausfuehrung und injiziert die notwendigen HTML Attribute.
    /// </summary>
    /// <param name="context">Der Kontext der aktuellen TagHelper Ausfuehrung.</param>
    /// <param name="output">Die resultierende HTML Ausgabe des TagHelpers.</param>
    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        output.TagName = "button";
        output.Attributes.SetAttribute("type", "button");
        if (!string.IsNullOrWhiteSpace(AriaLabel))
        {
            output.Attributes.SetAttribute("aria-label", AriaLabel);
        }
        if (IsActive)
        {
            output.Attributes.SetAttribute("aria-pressed", "true");
        }
        var brandColorHex = _skinProvider.GetPrimaryColor();
        var contrastClass = CalculateWcagContrastClass(brandColorHex);
        var existingClasses = output.Attributes.ContainsName("class")
            ? output.Attributes["class"].Value.ToString()
            : string.Empty;
        var finalClasses = $"btn {contrastClass} focus:ring-2 focus:ring-offset-2 {existingClasses}".Trim();
        output.Attributes.SetAttribute("class", finalClasses);
    }
    /// <summary>
    /// Berechnet die korrekte Textfarbe basierend auf der Hintergrundfarbe zur Einhaltung der Kontrastwerte.
    /// </summary>
    /// <param name="hexColor">Der Hex Farbwert der Hintergrundfarbe.</param>
    /// <returns>Die entsprechende Tailwind CSS Klasse fuer den Text.</returns>
    private string CalculateWcagContrastClass(string hexColor)
    {
        return "text-white";
    }
}
```

Listing.3.3 BFSG-A11y IViewLocalizer

```
@using Microsoft.AspNetCore.Mvc.Localization
@inject IViewLocalizer Localizer

<article
    class="ticket-card bg-white border-l-4 shadow-sm focus-within:ring-2"
    style="border-color: var(--brand-primary);"
    aria-labelledby="ticket-title-@Model.Id"
    aria-describedby="ticket-desc-@Model.Id"
    tabindex="0">

    <header class="flex justify-between items-center mb-2">
        <h3 id="ticket-title-@Model.Id" class="text-lg font-semibold text-gray-900">
            @Model.Title
```

```
        </h3>
        <span
            class="badge @Model.PriorityColor"
            role="status"
            aria-label="@Localizer["PriorityLevel"]: @Localizer[Model.PriorityName]">
            @Localizer[Model.PriorityName]
        </span>
    </header>

    <p id="ticket-desc-@Model.Id" class="text-sm text-gray-600 line-clamp-3">
        @Model.Description
    </p>

    <footer class="mt-4 flex items-center">
        <a11y-button
            aria-label="@Localizer["EditTicketAction", Model.Title]"
            is-active="false"
            class="hover:bg-gray-100 transition-colors">
            <i class="fa-solid fa-pen" aria-hidden="true"></i>
            <span class="sr-only">@Localizer["Edit"]</span>
        </a11y-button>
    </footer>
</article>
```

Listing.3.3 BFSG-A11y HTML Umsetzung

A.6 Benutzerhandbuch Tickets Please

1. Systemzugang

Der Zugriff auf das System erfolgt über den Webbrowser unter der bereitgestellten URL. Zur Anmeldung sind die geschäftliche E-Mail Adresse sowie das persönliche Kennwort erforderlich. Nach erfolgreicher Authentifizierung erfolgt die Weiterleitung zum zentralen Dashboard.

2. Dashboard Übersicht

Das Dashboard bietet eine Zusammenfassung der aktuellen Aktivitäten.

- Anzeige der dem Benutzer zugewiesenen Tickets.
- Übersicht über die Ticketprioritäten.
- Schnellzugriff auf zuletzt bearbeitete Vorgänge.
- Suchfunktion für die globale Ticketrecherche.

3. Ticketverwaltung

Ticket erstellen

- Betätigen Sie die Schaltfläche Neues Ticket.
- Geben Sie einen aussagekräftigen Titel ein.
- Beschreiben Sie das Anliegen im Textfeld.
- Wählen Sie eine Kategorie sowie die entsprechende Priorität aus.
- Speichern Sie den Vorgang ab.

Ticket bearbeiten

- Wählen Sie ein Ticket aus der Listenansicht aus.
- Nutzen Sie die Kommentarfunktion für die Kommunikation.
- Laden Sie bei Bedarf relevante Dateien oder Screenshots hoch.
- Ändern Sie den Status entsprechend dem Bearbeitungsfortschritt.

4. Workflow und Status

Tickets durchlaufen einen definierten Lebenszyklus.

- Neu: Das Ticket wurde erstellt und wartet auf Sichtung.
- In Bearbeitung: Ein Mitarbeiter arbeitet aktiv an der Lösung.
- Rückfrage: Weitere Informationen vom Ersteller sind notwendig.
- Gelöst: Die technische Lösung wurde bereitgestellt.
- Geschlossen: Der Vorgang ist final abgeschlossen.

5. Benutzereinstellungen

Über das Profilsymbol in der Kopfzeile gelangen Sie zu den persönlichen Einstellungen.

- Aktualisierung der Kontaktdaten.
- Verwaltung der Benachrichtigungsoptionen für Statusänderungen.
- Änderung des Systemkennworts.

6. Fehlerbehandlung

Sollten technische Probleme auftreten, nutzen Sie bitte den integrierten Support Link am Ende der Seite. Halten Sie die Ticketnummer oder die Fehlermeldung für eine schnelle Bearbeitung bereit.

A. 6.5 Test

A.7 Abnahmeprotokoll

Projektdaten

- **Projektbezeichnung:** Tickets Please
- **Projektverantwortlicher:** Tobias Boyke
- **Auftraggeber:** Beispiel GmbH
- **Datum der Abnahme:** 16.04.2026
- **Ort der Abnahme:** Neuss

Teilnehmer

- Tobias Boyke: Auftragnehmer
- Max Mustermann: Projektverantwortlicher Auftraggeber
- Erika Musterfrau: Qualitätsmanagement

Prüfprotokoll der Anforderungen

Anforderung ID	Beschreibung der Funktion	Status	Anmerkung
AF.01	Erstellung von Support Tickets	Bestanden	Validierung der Eingaben korrekt
AF.02	Zuweisung von Bearbeitern	Bestanden	Rollenprüfung erfolgreich
AF.03	Statusänderung über Workflow	Bestanden	Service Layer validiert Übergänge
AF.04	Suche und Filterung	Bestanden	Performance innerhalb der Vorgaben
NF.01	Einhaltung der Clean Architecture	Bestanden	Verifiziert durch NetArchTest
NF.02	UI Responsivität	Bestanden	Getestet mit Tailwind CSS 4.2
NF.03	Datensicherheit	Bestanden	DSGVO konforme Datenhaltung

A.8 Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe angefertigt und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Neuss, 16.04.2026 (Unterschrift Tobias Boyke)

A.9 Obligatorisches Rezept

Spaghetti all'Assassina (Die Mörder-Spaghetti)

Dieses Gericht wird traditionell direkt in der Pfanne in Öl und Tomatenmark geröstet, bis die Nudeln teilweise knusprig und fast verbrannt sind.

Zutaten:

- 320 g Spaghetti
- 150 g Tomatenmark
- 500 ml Tomatenpassata
- 800 ml Wasser
- 3 Knoblauchzehen
- 2 Getrocknete Chilischoten
- 100 ml Olivenöl
- Salz und Zucker

Zubereitung :

Tomatenbrühe: Passata, Wasser und 50 g Tomatenmark in einem Topf mischen, salzen, eine Prise Zucker hinzufügen und zum Kochen bringen.

Saucenansatz: In einer großen Eisenpfanne das Olivenöl stark erhitzen. Knoblauch und Chili darin anbraten, bis der Knoblauch goldbraun ist. Knoblauch entfernen.

Rösten: Das restliche Tomatenmark in das Öl geben und verrühren. Die ungekochten Spaghetti direkt in die Pfanne legen. Bei mittlerer Hitze anbraten, bis sie auf einer Seite braun und knusprig werden. Vorsichtig wenden.

Risotto-Methode: Sobald die Nudeln Röststellen haben, zwei Schöpfkellen der heißen Tomatenbrühe hinzugeben. Nicht rühren. Warten, bis die Flüssigkeit fast vollständig aufgesogen wurde und die Nudeln wieder anfangen zu braten.

Wiederholung: Diesen Vorgang wiederholen, bis die Spaghetti al dente sind. Die Nudeln sollten am Ende eine konzentrierte, dunkle Tomatenschicht und knusprige Stellen aufweisen.

Beilage: Fenchel-Orangen-Salat

Da die Spaghetti sehr intensiv, scharf und ölig sind, eignet sich ein frischer, säurebetonter Salat als Kontrast.

Zutaten:

- **2** Fenchelknollen
- **2** Orangen
- **1** Rote Zwiebel
- **3 EL** Olivenöl
- **1 EL** Weißer Balsamico
- Salz und schwarzer Pfeffer

Zubereitung:

1. Fenchel sehr fein hobeln.
2. Orangen filetieren und den Saft dabei auffangen.
3. Zwiebel in hauchdünne Ringe schneiden.
4. Aus Olivenöl, Essig, Orangensaft, Salz und Pfeffer ein Dressing rühren.
5. Alle Zutaten vermengen und das Fenchelgrün als Garnitur verwenden.